

ÜNİTE 4

Klasik Aramanın Ötesinde

Beyond Classical Search

Belgenin amacı: Bu çalışma, kullanıcı tarafından sağlanan AIMA 3. Baskı Ünite 4 metninin Türkçe anlatımını; ders sırasında kullanılabilecek açıklamalar, sayısal örnekler, özgün şemalar ve paylaşılan figürlerle birlikte tek bir çalışma notunda birleştirir. Bölüm sonundaki alıştırmalar Türkçeye çevrilmiş ve çözümleri açıklanmıştır.

Hazırlanan ders notu • Türkçe • Açıklamalı

İçindekiler

4 Klasik Aramanın Ötesinde

4.1 Yerel Arama Algoritmaları ve Optimizasyon Problemleri

4.1.1 Tepe Tırmanma Araması (Hill Climbing)

4.1.2 Benzetimli Tavlama (Simulated Annealing)

4.1.3 Yerel Işın Araması (Local Beam Search)

4.1.4 Genetik Algoritmalar

4.2 Sürekli Uzaylarda Yerel Arama

4.3 Deterministik Olmayan Eylemlerle Arama

4.4 Kısmi Gözlemlerle Arama

4.5 Çevrimiçi Arama Ajanları ve Bilinmeyen Ortamlar

4.6 Ünite Özeti ve Karşılaştırma Tabloları

Alıştırmalar 4.1-4.14: Türkçe Çeviri, Çözüm ve Açıklama

Sözlük Temel Kavramlar

4. KLASİK ARAMANIN ÖTESİNDE

Bu bölümde, önceki bölümde kullanılan basitleştirici varsayımlar gevşetilerek gerçek dünyaya daha yakın arama problemlerine geçilir. Bölüm 3; ortamın gözlemlenebilir, deterministik ve bilinen olduğu, çözümün de başlangıç durumundan hedefe götüren bir eylem dizisi olduğu problemleri ele almıştı. Burada bu varsayımların kaldırılmasıyla ne olduğunu inceliyoruz.

İlk olarak 4.1 ve 4.2 bölümleri, başlangıçtan hedefe giden yolları sistematik biçimde saklamak yerine bir ya da birkaç mevcut durumu değerlendirip değiştiren yerel arama algoritmalarını ele alır. Bu algoritmalar, hedefe hangi yoldan ulaşıldığının önemsiz; yalnızca son durumun kalitesinin önemli olduğu problemlerde özellikle uygundur. Bu ailede fiziksel tavlama sürecinden esinlenen benzetimli tavlama ve evrimsel biyolojiden esinlenen genetik algoritmalar bulunur.

4.3 ve 4.4 bölümlerinde deterministiklik ve tam gözlemlenebilirlik varsayımları gevşetilir. Ajan bir eylemin sonucunu kesin biçimde öngöremiyorsa, gerçekleştirecek her sonuca göre ne yapacağını planlamak zorundadır. Ortam kısmen gözlemlenebilir olduğunda ajan, içinde bulunabileceği durumların kümesini de izlemelidir. 4.5 ise durum uzayının başlangıçta bilinmediği ve ajanın hareket ederken keşfetmek zorunda olduğu çevrimiçi aramayı ele alır.

Bölümün ana sorusu: Bir ajan, yolun önemsiz olduğu optimizasyon problemlerinde, belirsiz eylemlerde, eksik gözlem altında ve hatta çevreyi hiç bilmediğinde nasıl arama yapabilir?

4.1 Yerel Arama Algoritmaları ve Optimizasyon Problemleri

Şimdiye kadar gördüğümüz arama algoritmaları arama uzayını sistematik biçimde araştırmak için bir veya daha fazla yolu bellekte tutar ve her noktada hangi alternatiflerin denendiğini kaydeder. Bir hedef bulunduğu, hedefe götüren yol da çözümün parçasıdır. Oysa pek çok gerçek problemde yol önemli değildir; önemli olan son durumda elde edilen düzenlemedir.

8-vezir problemi buna iyi bir örnektir: vezirlerin hangi sırayla yerleştirildiği önemli değildir, yalnızca son yerleşimin saldırısız olması önemlidir. Aynı özellik entegre devre tasarımı, fabrika yerleşimi, iş çizelgeleme, otomatik programlama, telekomünikasyon ağı optimizasyonu, araç rotalama ve portföy yönetimi gibi çok sayıda uygulamada görülür.

Yerel arama algoritmaları genellikle tek bir mevcut düğümle çalışır ve yalnızca o düğümün komşularına hareket eder. İzlenen yollar çoğunlukla saklanmaz. Bu nedenle iki önemli avantajları vardır: çok az bellek kullanırlar ve sistematik aramanın pratik olmadığı çok büyük ya da sürekli durum uzaylarında makul kalitede çözümler bulabilirler.

Yerel arama aynı zamanda saf optimizasyon problemlerinde kullanılır. Böyle problemlerde amaç, bir amaç fonksiyonuna göre en iyi durumu bulmaktır. Klasik anlamda hedef testi veya yol maliyeti bulunmak zorunda değildir. Örneğin evrimde üreme başarısı bir amaç fonksiyonu gibi düşünülebilir.

$$\text{En iyi durum} = \arg \max_s f(s) \quad \text{veya} \quad \arg \min_s h(s)$$

Durum Uzayı Manzarası

Yerel aramayı anlamak için durum uzayını bir arazi gibi düşünebiliriz. Yatay konum bir durumu, yükseklik ise amaç fonksiyonu ya da sezgisel maliyeti temsil eder. Yükseklik maliyetse amaç en derin vadiyi, yani küresel minimumu bulmaktır. Yükseklik kalite/amaç değeri ise amaç en yüksek tepeyi, yani küresel maksimumu bulmaktır. Bir maksimizasyon problemini minimizasyona çevirmek için işaret değiştirmek yeterlidir.

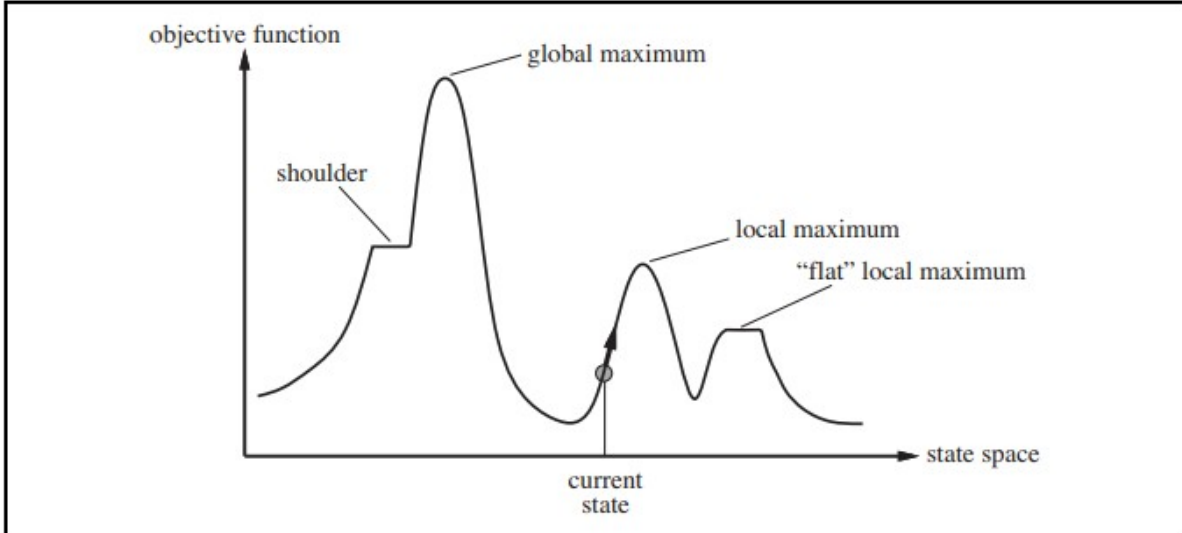


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. Hill-climbing search modifies the current state to try to improve it, as shown by the arrow. The various topographic features are defined in the text.

Şekil 4.1 - Bir boyutlu durum uzayı manzarası. Yükseklik amaç fonksiyonunu temsil eder; hedef küresel maksimumdur. Yerel maksimum, düz yerel maksimum ve omuz (shoulder) gibi yapılar hill climbing için tuzak oluşturabilir.

Kavram farkı: Local maximum yalnızca yakın komşularından daha iyidir; global maximum tüm durum uzayındaki en iyi noktadır. Plateau düz bir bölgedir. Shoulder ise düz görünse de bir süre yatay hareketten sonra yükselmenin devam edebildiği plato türüdür.

4.1.1 Tepe Tırmanma Araması (Hill-Climbing Search)

Steepest-ascent hill climbing sürekli olarak değeri artıran yöne hareket eder. Her adımda mevcut düğüm en iyi komşuyla değiştirilir. Hiçbir komşu mevcut durumdan daha iyi değilse algoritma durur. Arama ağacı tutmadığı için bellekte yalnızca mevcut durum ve onun amaç/sezgisel değeri yeterlidir. Ayrıca yalnızca hemen komşulara bakar; ileride ne olacağını önceden araştırmaz.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current <- MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor <- current durumunun en yüksek değerli ardılı
    if neighbor.VALUE <= current.VALUE then
      return current.STATE
    current <- neighbor
```

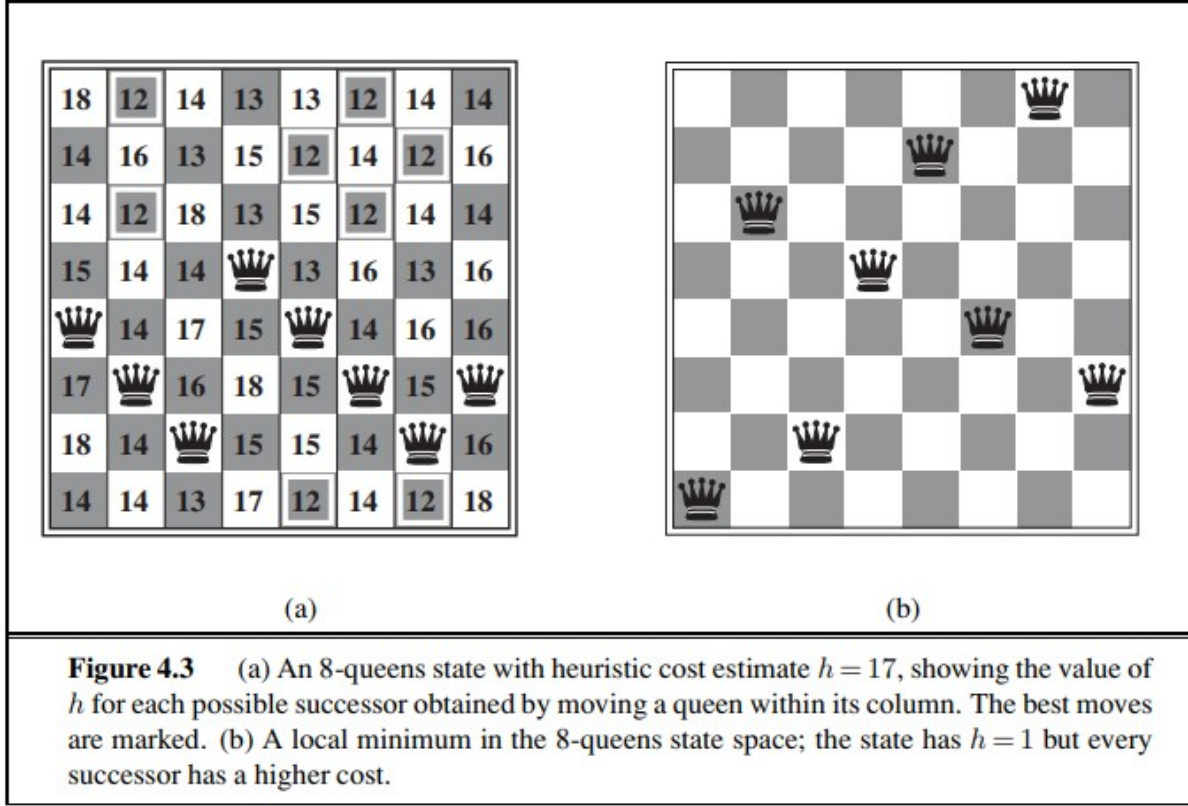
Şekil 4.2 - Hill-climbing algoritmasının sadeleştirilmiş Türkçe sözde kodu.

Kitabın benzetmesiyle hill climbing, yoğun sis altında ve hafıza kaybı yaşarken Everest'in zirvesini bulmaya çalışmaya benzer: ajan yalnızca bulunduğu yerin çevresini görebilir, geçmiş yolu saklamaz ve uzak bölgelerde daha iyi tepeler olup olmadığını bilemez.

8-Vezir Üzerinde Hill Climbing

Yerel aramada 8-vezir problemi complete-state biçiminde ifade edilir: her sütunda tam bir vezir vardır. Bir ardıl oluşturmak için tek bir vezir kendi sütununda başka bir satıra taşınır. Her vezir için 7 alternatif, 8 vezir için toplam $8 \times 7 = 56$ ardıl durum vardır.

Sezgisel maliyet h , birbirine saldıran vezir çiftlerinin sayısıdır. Mükemmel bir çözümde $h = 0$ olur. Şekil 4.3(a)'daki başlangıç durumunda $h = 17$ 'dir. Karelerin üzerindeki sayılar ilgili sütundaki vezirin o kareye taşınması durumunda oluşacak yeni h değerini gösterir. En düşük değerli ardıllar en iyi hareketlerdir.

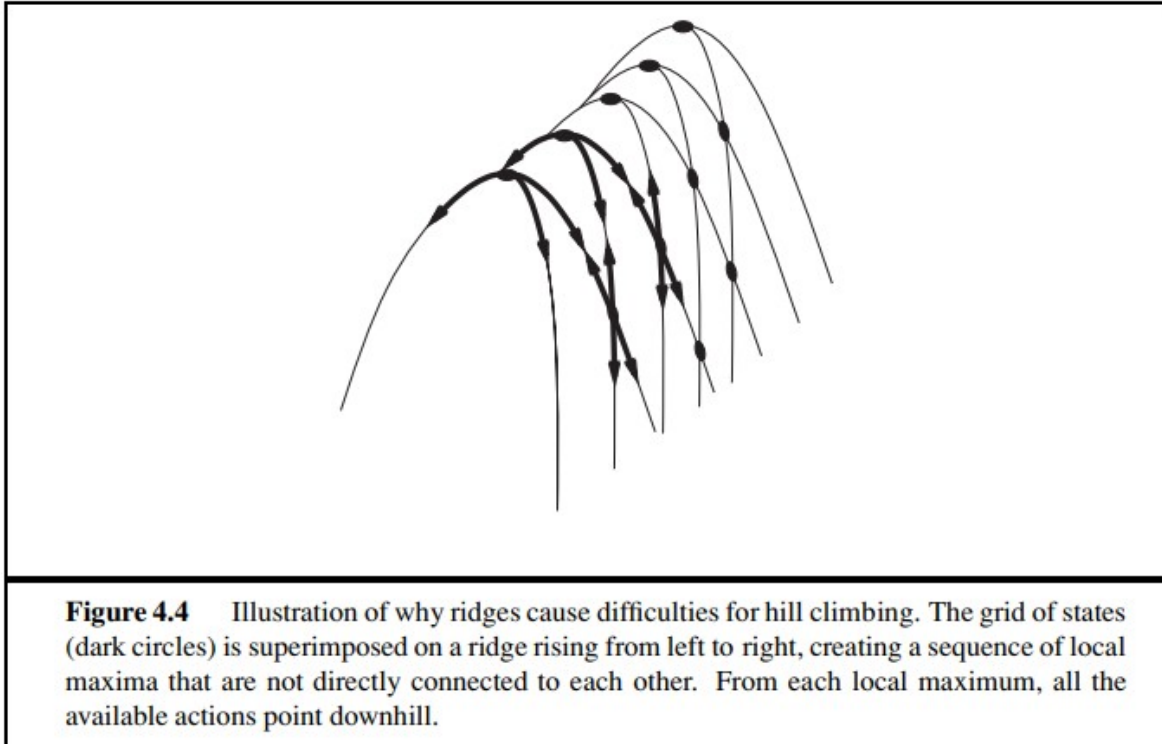


Şekil 4.3 - (a) $h=17$ olan 8-vezir durumu ve her olası hareketin maliyeti. (b) $h=1$ olduğu halde bütün tek-vezir hareketlerinin daha kötü olduğu bir yerel minimum.

Şekil 4.3(b) hill climbing'in temel zayıflığını gösterir. $h=1$ olduğundan çözüme çok yakınız, fakat her tek vezir hareketi h değerini artırır. Algoritma yalnızca iyileştirici hamleleri kabul ediyorsa burada durur. Maliyet açısından bu bir local minimum; değeri maksimize eden eşdeğer bakışta local maximumdur.

Hill Climbing'in Takıldığı Yapılar

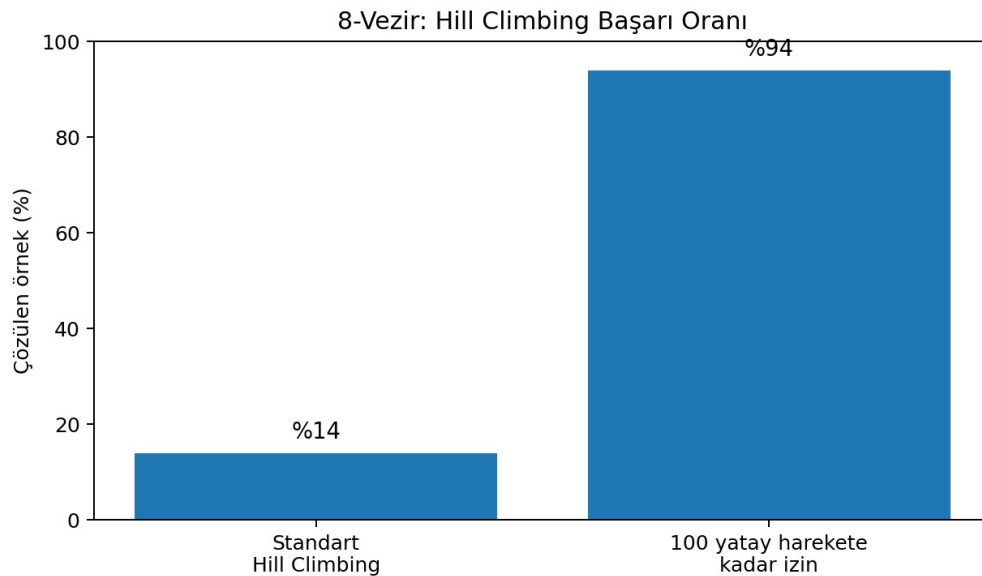
- Yerel maksimum/minimum: Yakındaki bütün komşulardan iyi, fakat global optimum olmayan nokta.
- Ridge (sırt): En iyi yönde ilerlemek için hareketlerin birleşimi gerekir; tek adımlık komşuluklar sırt boyunca doğrudan ilerlemeyi sağlayamaz.
- Plateau (plato): Çok sayıda komşu aynı değere sahiptir. Flat local maximum çıkışsızdır; shoulder ise yatay hareketten sonra tekrar yükselmeye izin verir.



Şekil 4.4 - Sırtların hill climbing için neden zor olduğu. Ayrık durum noktaları sırtın tepe çizgisine tam oturmadığı için her yerel tepeden izin verilen hamlelerin tümü aşağı yönlü görünebilir.

Standart steepest-ascent hill climbing, rastgele üretilmiş 8-vezir durumlarında yaklaşık %14 oranında çözüm bulur ve %86 oranında takılır. Başarılı olduğunda ortalama yaklaşık 4, takıldığında yaklaşık 3 adım sürer. Bu, yaklaşık $8^8 = 16.777.216$ durumlu bir uzay için son derece hızlıdır; ancak başarı oranı düşüktür.

Plato üzerinde aynı değerli komşulara geçmeye sideways move (yatay hareket) denir. Sınırsız yatay harekete izin vermek döngü yaratabilir; bu nedenle örneğin en fazla 100 ardışık yatay hareket sınırı konabilir. 8-vezirde bu değişiklik başarı oranını yaklaşık %14'ten %94'e çıkarır; fakat başarılı arama başına adım sayısı da artar.



Grafik - 8-vezir probleminde standart hill climbing ile yatay hareket sınırı eklenmiş sürümün kitapta verilen yaklaşık başarı oranları.

Hill Climbing Varyantları

Stochastic hill climbing, yükselen hareketler arasından rastgele seçim yapar; daha dik iyileştirmelere daha yüksek olasılık verilebilir. Genellikle steepest-ascent'e göre daha yavaş yakınsar, fakat bazı manzaralarda daha iyi çözümler bulabilir.

First-choice hill climbing, ardılları rastgele üretir ve mevcut durumdan daha iyi ilk ardılı bulduğu anda onu kabul eder. Özellikle bir durumun binlerce ardılı olduğunda tüm ardılları hesaplamak yerine yararlıdır.

Random-restart hill climbing, başarısız olduğunda yeni bir rastgele başlangıç durumu seçip yeniden çalıştırır. Bir aramanın başarı olasılığı p ise beklenen deneme sayısı $1/p$ 'dir. 8-vezirde $p \approx 0,14$ olduğunda yaklaşık 7 deneme gerekir. Yatay hareketle $p \approx 0,94$ ise ortalama yaklaşık 1,06 deneme yeterlidir. Bu nedenle random-restart hill climbing 8-vezir için son derece etkilidir.

$$\text{Beklenen yeniden başlatma sayısı} = 1 / p$$

Ders örneği: Bir hill-climbing çalışmasının başarı olasılığı %20 ise, ortalama $1/0,20 = 5$ çalışmada bir çözüm beklenir. Bu, yaklaşık 4 başarısız + 1 başarılı deneme anlamına gelir.

4.1.2 Benzetimli Tavlama (Simulated Annealing)

Hiçbir zaman aşağı yönlü, yani değeri azaltan veya maliyeti artıran hamle yapmayan hill climbing local optimuma takılabildiği için tamamlanmış bir yöntem değildir. Öte yandan tamamen rastgele yürüyüş local optimumlardan kaçabilir ama son derece verimsizdir. Benzetimli tavlama bu iki yaklaşımı birleştirir.

Metalurjide tavlama, bir malzemeyi yüksek sıcaklığa çıkarıp daha sonra yavaşça soğutarak düşük enerjili düzenli bir yapıya ulaştırma sürecidir. Arama benzetmesinde en derin çukura bir top yerleştirmek isteriz. Yüzeyi hiç sarsmazsak top ilk yerel çukurda kalabilir; başlangıçta güçlü, sonra giderek daha zayıf sarsarsak top küçük çukurlardan çıkabilir ve soğuma ilerledikçe iyi bir bölgede kararlı hale gelir.

Simulated annealing en iyi komşuyu seçmek yerine rastgele bir komşu seçer. Hareket daha iyiye her zaman kabul edilir. Daha kötüyse belirli bir olasılıkla yine kabul edilir. Kötüleşme miktarı arttıkça ve sıcaklık T azaldıkça bu olasılık düşer.

$$P(\text{kötü hareketi kabul}) = \exp(\Delta E / T), \quad \Delta E < 0$$

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current <- başlangıç durumu
  for t = 1 to sonsuz do
    T <- schedule(t)
    if T = 0 then return current
    next <- current'ın rastgele bir ardılı
    ΔE <- next.VALUE - current.VALUE
    if ΔE > 0 then current <- next
    else current <- next only with probability exp(ΔE/T)
```

Şekil 4.5 - Benzetimli tavlama sözde kodu.

Sezgi: Yüksek T = keşif ve risk alma. Düşük T = istikrar ve hill climbing'e yaklaşma. Soğutma çok hızlı olursa algoritma erken donar; çok yavaş olursa daha kaliteli çözüm bulabilir ama uzun sürer.

Sıcaklık programı yeterince yavaş düşürülürse global optimumu bulma olasılığı teorik olarak 1'e yaklaşabilir. Simulated annealing ilk olarak 1980'lerin başlarında VLSI yerleşimi için yoğun biçimde kullanılmış; daha sonra fabrika çizelgeleme ve başka büyük optimizasyon problemlerine uygulanmıştır.

4.1.3 Yerel Işın Araması (Local Beam Search)

Local beam search tek bir durum yerine aynı anda k durum tutar. Başlangıçta k rastgele state üretilir; her adımda bu k state'in bütün ardılları oluşturulur. Herhangi biri hedefse arama biter, değilse tüm havuzdan en iyi k ardıl seçilir ve süreç tekrar edilir.

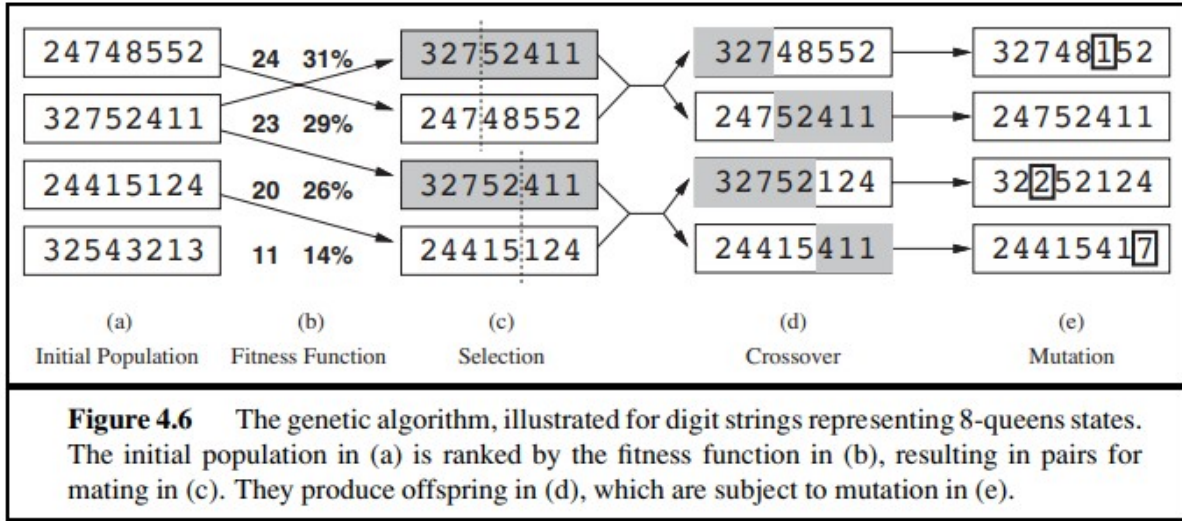
Bu yöntem k random-restart aramasını paralel yürütmekten farklıdır. Random restart'ta süreçler bağımsızdır. Local beam search'te ise iyi ardıllar üreten durumlar diğer arama kollarının kaynaklarını kendi bölgelerine çeker; böylece başarısız bölgeler terk edilip umut veren bölgeler üzerinde yoğunlaşılır.

Temel local beam search çeşitlilik kaybına uğrayabilir: k state kısa sürede aynı küçük bölgeye toplanabilir. Stochastic beam search bunu azaltmak için en iyi k state'i deterministik biçimde almak yerine adayları değerleriyle orantılı olasılıklarla seçer. Bu, doğal seçim fikrine yaklaşıp.

4.1.4 Genetik Algoritmalar

Genetik algoritma, successor state'lerin tek bir ebeveyni değiştirerek değil, iki ebeveyni birleştirerek oluşturulduğu stochastic beam search varyantıdır. Doğal seçim analojisi korunur; ancak artık eşeysiz değil eşeyli üremeye benzer bir mekanizma vardır.

GA, k rastgele bireyden oluşan bir population ile başlar. Her birey sonlu bir alfabe üzerindeki diziyle temsil edilir. 8-vezirde durum 8 basamakla gösterilebilir: örneğin 24748552 dizisinin i . basamağı i . sütundaki vezirin satırını belirtir. Alternatif olarak 24 bitlik bir kodlama da mümkündür.



Şekil 4.6 - 8-vezir dizileri üzerinde genetik algoritma: başlangıç popülasyonu, fitness, seçim, crossover ve mutation.

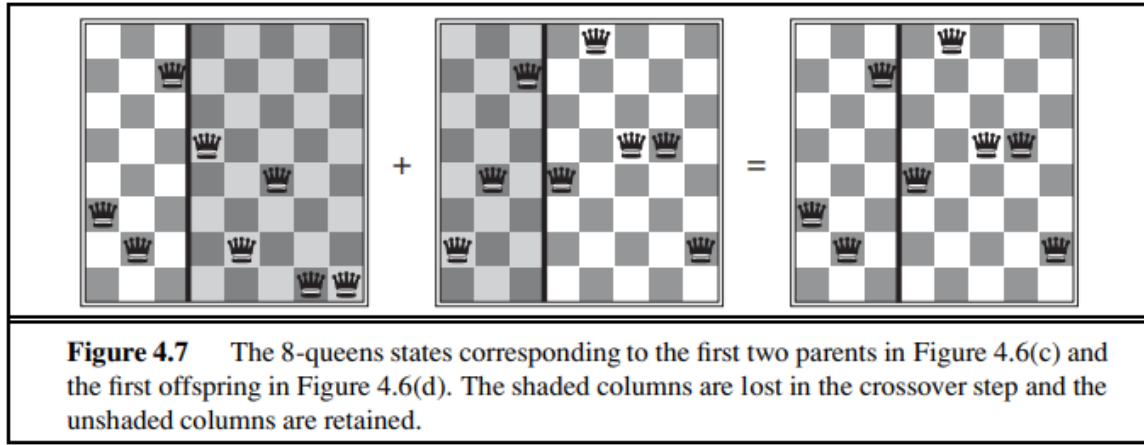
Fitness function daha iyi durumlara daha yüksek değer vermelidir. 8 vezir arasında toplam 8 choose 2 = 28 çift vardır; bu nedenle birbirine saldırmayan çiftlerin sayısını fitness olarak kullanabiliriz. Kusursuz bir çözümün fitness değeri 28'dir. Şekilde dört bireyin skorları 24, 23, 20 ve 11'dir.

$$\text{Toplam fitness} = 24 + 23 + 20 + 11 = 78$$

$$P(\text{ilk birey}) = 24 / 78 \approx \%31$$

Seçim olasılığı fitness ile orantılı olduğundan iyi bireylerin üreme ihtimali daha yüksektir; ancak düşük fitness'lı bireyler tamamen dışlanmayabilir. Aynı birey iki kez seçilebilir, bazı bireyler hiç seçilmeyebilir.

Crossover için rastgele bir kesme noktası seçilir. Örneğin 327|52411 ile 247|48552 ebeveynleri çaprazlanırsa çocuklardan biri 327|48552 = 32748552 olur. İkinci çocuk ters parçaları kullanarak 24752411 olabilir.



Şekil 4.7 - Şekil 4.6'daki ilk iki ebeveyn ve ilk çocuğun 8-vezir tahtası. Crossover'da bazı sütunlar bir ebeveynden, kalanlar diğerinden korunur.

Mutation aşamasında her konum küçük ve bağımsız bir olasılıkla değiştirilebilir. 8-vezirde bu, rastgele bir veziri kendi sütununda başka bir satıra taşımaya karşılık gelir. Mutation yeni çeşitlilik sağlar; crossover ise mevcut iyi yapı parçalarını yeniden birleştirir.

```
function GENETIC-ALGORITHM(population, FITNESS-FN)
  repeat
    new_population <- boş
    for i = 1 to SIZE(population) do
      x <- RANDOM-SELECTION(population, FITNESS-FN)
      y <- RANDOM-SELECTION(population, FITNESS-FN)
      child <- REPRODUCE(x, y)
      küçük bir olasılıkla child <- MUTATE(child)
      child'ı new_population'a ekle
    population <- new_population
  until yeterince uygun birey bulundu veya süre doldu
  return population içindeki en iyi birey

function REPRODUCE(x, y)
  n <- LENGTH(x)
  c <- 1..n arasında rastgele kesme noktası
  return x[1..c] + y[c+1..n]
```

Şekil 4.8 - Genetik algoritmanın Türkçeleştirilmiş sözde kodu.

Schema ve Temsilin Önemi

Genetik algoritma teorisinde schema, bazı konumların sabit, bazılarının önemsiz bırakıldığı bir kalıptır. Örneğin 246***** şeması ilk üç veziri sırasıyla 2., 4. ve 6. satırlarda bulunan tüm 8-vezir durumlarını temsil eder. 24613578 bu şemanın bir örneğidir.

Bir schema'nın örneklerinin ortalama fitness'ı popülasyon ortalamasından yüksekse, bu şemaya uyan bireylerin sayısı zamanla artma eğilimindedir. Crossover'ın avantajı, bağımsız olarak yararlı hale gelmiş büyük blokları birleştirebilmesidir. Fakat bu avantaj, genetik kod içindeki komşu konumların gerçekten anlamlı bir yapı oluşturmalarına bağlıdır.

Bu nedenle representation çok önemlidir. 8 rakamlık kodlamada crossover bir vezir konumları bloğunu koruyabilir. 24 bitlik kodlamada kesme noktası bir rakamın bitleri arasına düşebilir; bu durumda o rakam neredeyse rastgele mutasyona uğramış gibi olur. Başarılı GA tasarımı dikkatli temsil mühendisliği gerektirir.

Evrin ve Arama

Darwin'in doğal seçim kuramının temel fikri basittir: üreme sırasında varyasyonlar oluşur ve üreme başarısını artıran varyasyonlar sonraki nesillerde daha yüksek oranda korunur. Mendel daha sonra

kalıtımın olasılıksal yasalarını ortaya koymuş, Watson ve Crick DNA molekülünün yapısını açıklamıştır. DNA'daki varyasyon yalnızca nokta mutasyonlarından değil, uzun parçaların crossover ile birleşmesinden de kaynaklanır.

Gerçek biyolojik evrim, standart genetik algoritmalarından çok daha zengindir: DNA parçalarının ters dönmesi, kopyalanması, taşınması, virüslerle aktarılması ve transpozonlar gibi mekanizmalar bulunur. Ayrıca biyolojide genom, kendi kopyalanma ve organizmaya çevrilme mekanizmalarını da kodlar; genetik algoritmalarda ise bu mekanizma ayrı bir programdır.

Lamarck, yaşam sırasında kazanılan özelliklerin yavrulara aktarılabileceğini öne sürmüştür; bu mekanizma genel olarak doğada gözlenmez. Baldwin ise öğrenilmiş davranışların, genetik aktarım olmaksızın seçim baskısını değiştirerek evrimi hızlandırabileceğini göstermiştir. Bilgisayar simülasyonları uygun koşullarda Baldwin etkisinin ortaya çıkabildiğini doğrular.

Algoritma	Kaç state tutar?	Kötü harekete izin?	Temel risk / avantaj
Hill Climbing	1	Hayır	Hızlıdır; local optimuma takılabilir.
Simulated Annealing	1	Evet, sıcaklığa bağlı	Local optimumdan kaçabilir.
Local Beam Search	k	Dolaylı olarak	Birden fazla bölgeyi birlikte izler; çeşitlilik kaybı olabilir.
Genetic Algorithm	Popülasyon	Crossover/mutation ile	İyi yapı parçalarını birleştirebilir; temsil kritik.

4.2 Sürekli Uzaylarda Yerel Arama

Gerçek dünyanın birçok ortamı sürekli dir. Sürekli state ve action uzaylarında, çoğu klasik algoritma sonsuz dallanma faktörü nedeniyle doğrudan kullanılamaz. First-choice hill climbing ve simulated annealing gibi rastgele ardıl üretebilen yöntemler ise sürekli uzaylara daha doğal biçimde uyarlanabilir.

Üç Havaalanı Yerleştirme Örneği

Romanya'da üç yeni havaalanını, her şehrin en yakın havaalanına olan karesel uzaklıklarının toplamını minimize edecek şekilde yerleştirmek istediğimizi düşünelim. Havaalanlarının koordinatları (x_1, y_1) , (x_2, y_2) , (x_3, y_3) olduğundan durum uzayı altı boyutludur. Genel olarak sürekli bir state, n boyutlu x değişken vektörüyle tanımlanır.

C_i , mevcut durumda en yakın havaalanı i olan şehirlerin kümesi olsun. C_i kümeleri sabit kaldığı küçük bir çevrede amaç fonksiyonu şu biçimdedir:

$$f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_i \sum_{c \in C_i} [(x_i - x_c)^2 + (y_i - y_c)^2]$$

Bu ifade yerel olarak doğrudur; global olarak C_i kümeleri state'e bağlı olarak kesintili biçimde değişebilir. Havaalanlarından biri hareket ettiğinde bir şehir başka havaalanına daha yakın hale gelebilir ve hangi toplamın içinde yer aldığı değişebilir.

Ayrıklaştırma ve Gradyan

Sürekli problemi ayrık hale getirmenin basit yolu her adımda tek bir koordinatı $\pm \delta$ kadar değiştirmektir. Altı değişken olduğundan her state için 12 komşu elde edilir ve önceki yerel arama algoritmaları uygulanabilir. Stochastic hill climbing ve simulated annealing ise rastgele uzunluğu δ olan vektörler üreterek sürekli uzayda doğrudan çalışabilir.

Birçok yöntem yüzeyin gradyanını kullanır. Gradyan ∇f , en dik artış yönünü ve büyüklüğünü veren vektördür:

$$\nabla f = (\partial f / \partial x_1, \partial f / \partial y_1, \partial f / \partial x_2, \partial f / \partial y_2, \partial f / \partial x_3, \partial f / \partial y_3)$$

Yerel gradyan biliniyorsa steepest-ascent güncellemesi şu şekilde yapılır:

$$x \leftarrow x + \alpha \nabla f(x)$$

Buradaki α step size, yani adım büyüklüğüdür. α çok küçükse çok fazla adım gerekir; çok büyükse optimum nokta aşılabılır. Line search, mevcut gradyan yönünde α 'yı çoğunlukla tekrar tekrar büyütüp amaç fonksiyonu kötüleşmeye başlayana kadar ilerleyerek uygun adım büyüklüğünü arar.

Makine öğrenmesi bağlantısı: Gradient descent'te kullanılan learning rate ile burada kullanılan step size aynı temel fikirdir. Maksimizasyon için $+\alpha \nabla f$, minimizasyon için $-\alpha \nabla f$ yönünde ilerlenir.

Deneyisel Gradyan ve Newton-Raphson

Amaç fonksiyonunun türevlenebilir kapalı formu bulunmadığında, her koordinatta küçük artış ve azalışlar denenerek empirical gradient tahmin edilebilir. Bu yaklaşım, ayrıklaştırılmış durum uzayındaki steepest-ascent hill climbing ile yakından ilişkilidir.

Newton-Raphson yöntemi başlangıçta $g(x)=0$ biçimindeki denklemlerin köklerini bulmak için geliştirilmiştir:

$$x \leftarrow x - g(x) / g'(x)$$

Bir maksimum veya minimumda gradyanın sıfır olmasını istediğimiz için $g(x)=\nabla f(x)$ alınabilir. Çok boyutlu güncelleme:

$$x \leftarrow x - H_f(x)^{-1} \nabla f(x)$$

$H_f(x)$, ikinci türevlerden oluşan Hessian matrisidir. Newton yöntemi yerel yüzeyi kuadratik bir yüzeyle yaklaşıklar ve onun optimumuna doğru büyük, bilgi açısından zengin bir adım atabilir. Fakat yüksek boyutta Hessian'ın n^2 elemanını hesaplamak ve matris tersini almak pahalıdır; bu nedenle çok sayıda yaklaşık Newton yöntemi geliştirilmiştir.

Kısıtlı ve Konveks Optimizasyon

Yerel maksimum, ridge ve plateau problemleri sürekli uzaylarda da vardır; random restart ve simulated annealing yine yararlı olabilir. Ayrıca çözümlerin belirli sert kısıtları sağlaması gereken problemler constrained optimization olarak adlandırılır. Havaalanlarının ülke sınırı içinde ve kara üzerinde olması buna örnektir.

Kısıtların doğrusal eşitsizlikler oluşturduğu ve amaç fonksiyonunun da doğrusal olduğu özel sınıf linear programming'dir. Doğrusal programlamanın zaman karmaşıklığı değişken sayısında polinomiktir ve en yaygın incelenen optimizasyon sınıflarından biridir.

Convex set, içindeki herhangi iki noktayı birleştiren doğru parçasının tamamının yine küme içinde kaldığı kümedir. Convex optimization'da kısıt bölgesi konvektir ve amaç fonksiyonu uygun biçimde konvektir. Konveks minimizasyonda kötü yerel minimum bulunmaması, yani yerel minimumun global minimum olması temel avantajdır.

Basit örnek: Minimize $f(x,y)=x^2+y^2$, kısıt $x+y \geq 1$. Amaç yüzeyi konvektir, kısıt bölgesi de konvektir. Bu nedenle bulunan yerel minimum aynı zamanda global minimumdur.

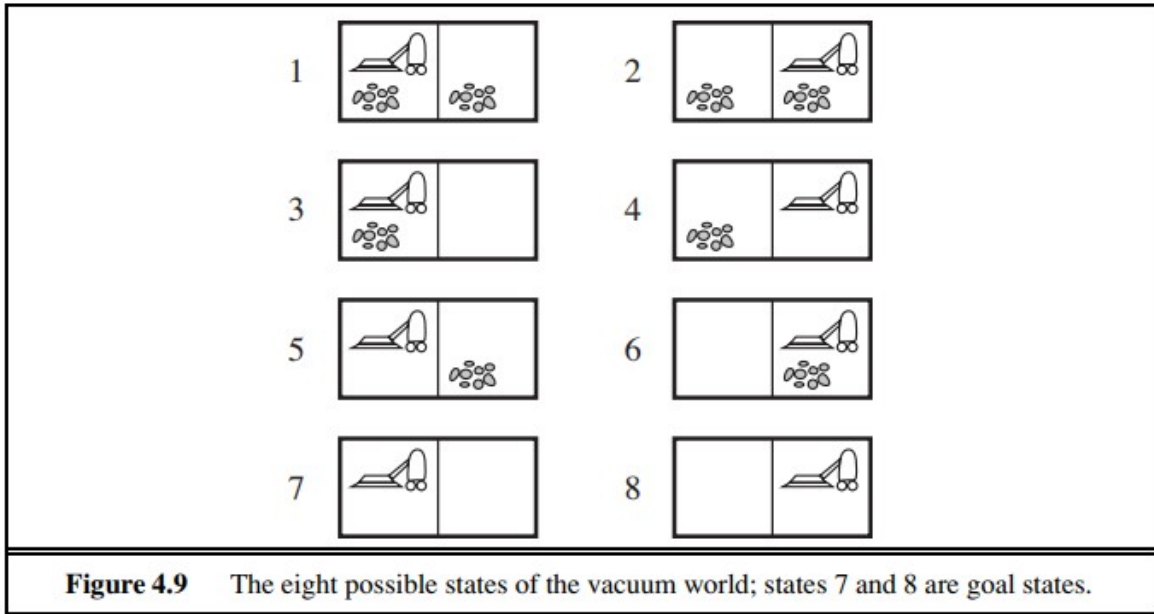
4.3 Deterministik Olmayan Eylemlerle Arama

Bölüm 3'te ajan, ortamın tam gözlemlenebilir ve deterministik olduğunu, ayrıca her eylemin etkisini bildiğini varsayıyordu. Bu nedenle herhangi bir eylem dizisinin hangi state'e götüreceğini tam olarak hesaplayabiliyor ve her zaman hangi state'te olduğunu biliyordu.

Ortam kısmen gözlemlenebilir veya nondeterministic olduğunda percept'ler tekrar önem kazanır. Kısmi gözlemde percept, mümkün state kümesini daraltır. Nondeterministic ortamda ise percept, eylemin olası sonuçlarından hangisinin gerçekten gerçekleştiğini bildirir. Gelecekte alınacak percept önceden kesin bilinmediği için gelecekte yapılacak eylemler de o percept'e bağlı olacaktır.

Bu nedenle çözüm artık tek bir eylem dizisi değil, koşullara göre ne yapılacağını belirleyen contingency plan veya strategy'dir.

4.3.1 Düzensiz Elektrikli Süpürge Dünyası



Şekil 4.9 - Süpürge dünyasının sekiz fiziksel durumu. 7 ve 8 numaralı state'lerde her iki kare de temizdir ve bunlar hedef durumlarıdır.

Normal süpürge dünyasında Left, Right ve Suck olmak üzere üç eylem vardır. Hedef tüm kiri temizlemektir. Deterministik ve tam bilinen durumda başlangıç state 1 ise [Suck, Right, Suck] dizisi hedef state 8'e ulaşır.

Erratic vacuum world'de Suck eylemi güçlü ama düzensizdir: kirli karede kullanıldığında o kareyi temizler ve bazen yan kareyi de temizler; temiz karede kullanıldığında ise bazen halıya yeniden kir bırakabilir. Bu nedenle RESULT tek state yerine olası state'ler kümesini döndüren RESULTS işlevine genellenir.

$$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$$

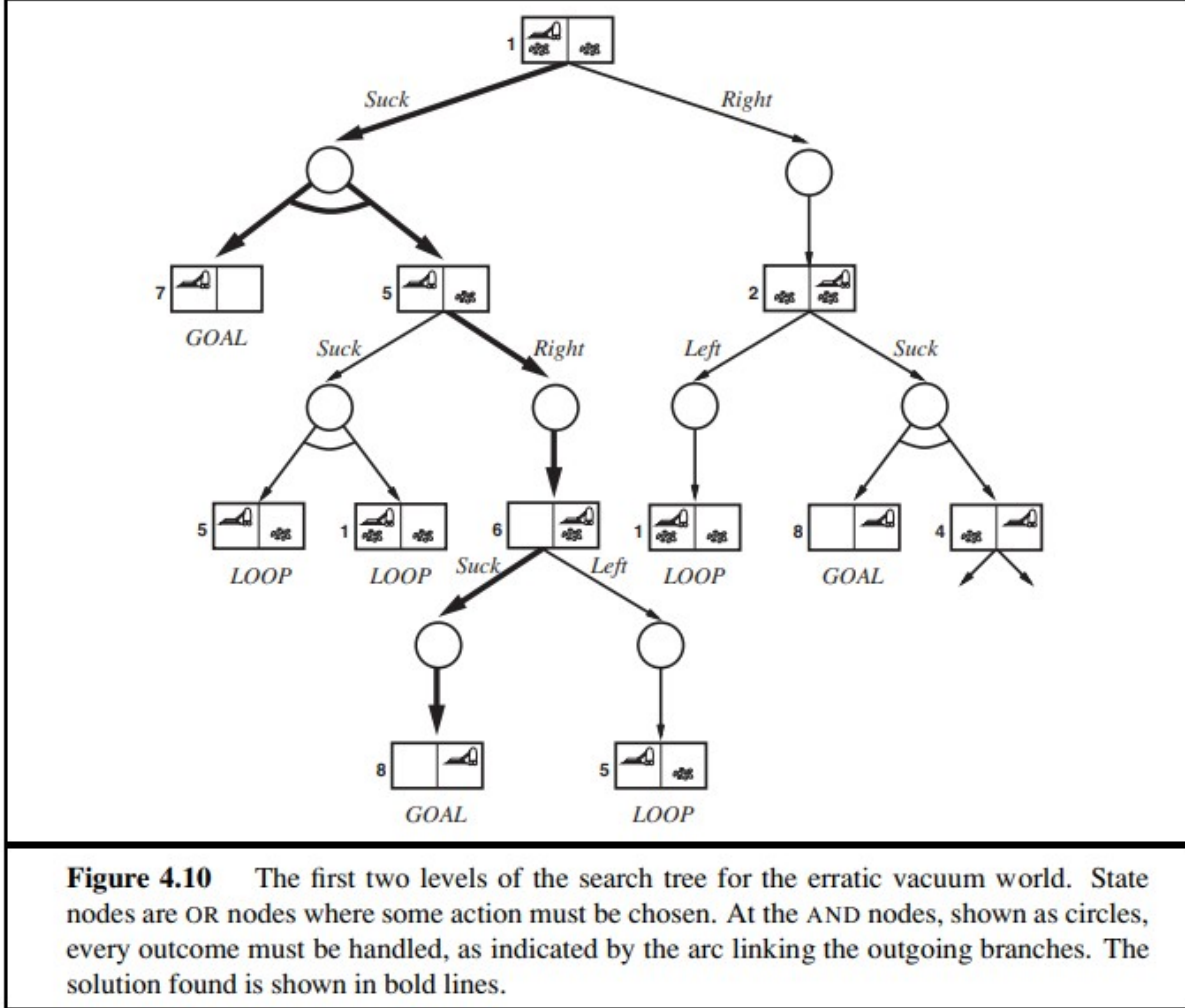
State 1'den Suck sonrası yalnızca sol kare temizlenirse state 5, iki kare birden temizlenirse state 7 oluşur. Başlangıç state 1 için tek bir sabit eylem dizisi artık yeterli değildir. Koşullu plan şu olabilir:

```
Suck
if State = 5 then
    Right
    Suck
else
    hiçbir şey yapma
```

Koşullu plan - İlk Suck sonucuna göre sonraki eylem seçilir.

4.3.2 AND-OR Arama Ağaçları

Deterministik ortamda dallanma yalnızca ajanın hangi action'ı seçtiğinden kaynaklanır. Bu state düğümlerine OR node denir: Left OR Right OR Suck seçeneklerinden birini seçmek yeterlidir. Nondeterministic ortamda ise bir action'ın sonucunu çevre seçer; bu sonuç düğümleri AND node'dur çünkü ajan action'ın bütün olası sonuçları için bir çözüm hazırlamak zorundadır.



Şekil 4.10 - Erratic vacuum world için AND-OR arama ağacının ilk seviyeleri. Dikdörtgen state düğümleri OR, daireler AND düğümleridir. Kalın dallar bulunan çözüm planını gösterir.

Bir AND-OR çözümü şu koşulları sağlayan bir alt ağaçtır: her yaprak hedef olmalı; her OR düğümünde tam bir eylem seçilmeli; her AND düğümünde ise bütün olası sonuç dalları plana dahil edilmelidir. Bu yapı, if-then-else içeren koşullu planla birebir eşleşir.

```
function AND-OR-GRAPH-SEARCH(problem)
  return OR-SEARCH(problem.INITIAL-STATE, problem, [])

function OR-SEARCH(state, problem, path)
  if GOAL-TEST(state) then return boş plan
  if state path üzerinde ise return failure
  for each action in ACTIONS(state) do
    plan <- AND-SEARCH(RESULTS(state, action), problem, [state | path])
    if plan != failure then return [action | plan]
  return failure

function AND-SEARCH(states, problem, path)
  for each si in states do
```

```

plan1 <- OR-SEARCH(si, problem, path)
if plan1 = failure then return failure
return koşullu plan: if s1 then plan1 else if s2 then plan2 ...

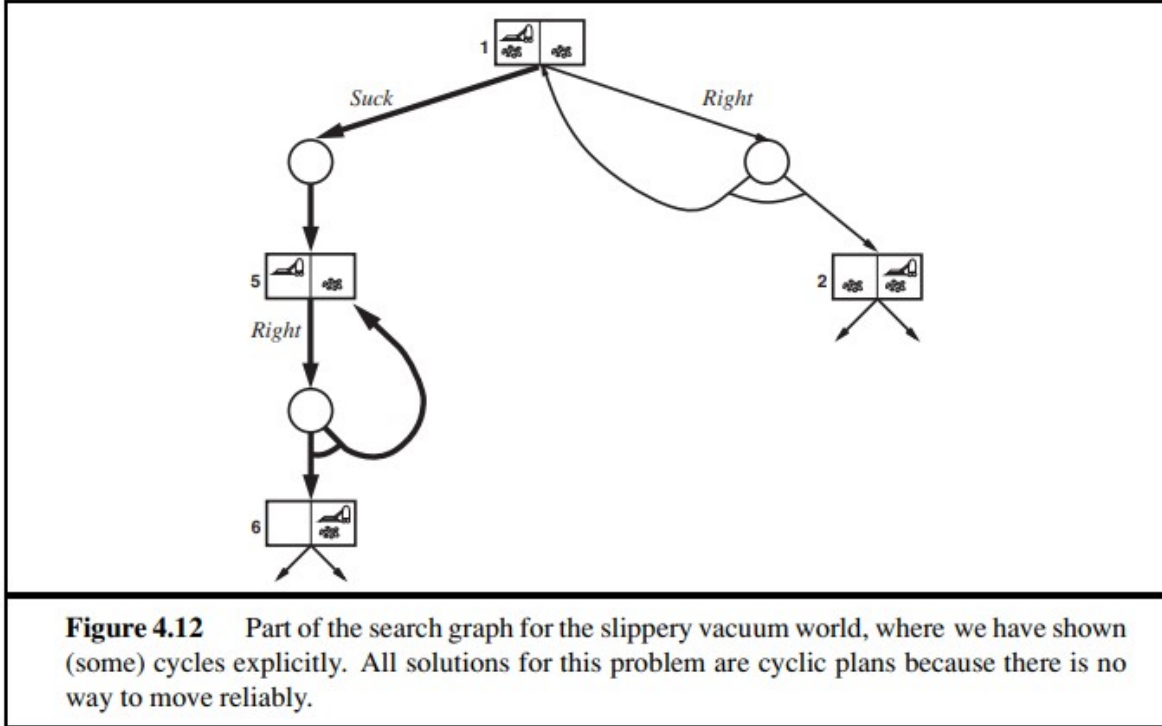
```

Şekil 4.11 - Nondeterministic ortamlar için AND-OR graph search sözde kodu.

Algoritma path üzerinde tekrar eden bir state görürse bu dalı failure kabul ederek döngüsüz plan arar. Bu, o state'ten hiçbir çözüm olmadığı anlamına gelmez; yalnızca çözüm döngüsüz olacaksa aynı state'in önceki kopyasından araştırmanın yeterli olduğunu belirtir. Sonlu state space'de her yol hedefe, çıkmaza veya tekrara ulaşacağı için bu kontrol sonlanmayı sağlar.

4.3.3 Dene, Olmazsa Tekrar Dene: Döngüsel Planlar

Slippery vacuum world, normal süpürge dünyasına benzer; tek fark hareket action'larının bazen başarısız olup aynı konumda bırakmasıdır. Örneğin state 1'de Right sonucu {1,2} olabilir. Böyle bir dünyada sonlu uzunlukta döngüsüz bir plan, Right'ın mutlaka başarılı olacağını garanti edemez.



Şekil 4.12 - Kaygan süpürge dünyasının arama grafiğinden bir bölüm. Hareket başarısızlığı aynı state'e geri dönen döngüler oluşturur; tüm çözümler döngüsel plan olmak zorundadır.

Çözüm, Right başarılı olana kadar tekrar etmektir. Planın bir bölümüne label verilip daha sonra o label'a geri dönülebilir:

```

Suck
L1: Right
if State = 5 then
  goto L1
else
  Suck

```

Döngüsel plan - "State 5 iken Right yap" biçimindeki while döngüsüne eşdeğerdir.

Bir cyclic planın çözüm sayılabilmesi için her yaprak hedef olmalı ve planın her noktasından bir hedef yaprağına ulaşılabilmelidir. Ayrıca nondeterministic action'ın olası sonuçlarının zaman içinde gerçekleşeceğine dair uygun bir fairness varsayımı gerekir. Zar atmak gibi gerçek rastlantısal süreçlerde bu makul olabilir; fakat yanlış otel kartı gibi gizli, kalıcı bir neden varsa aynı action'ı sonsuza kadar denemek doğru model değildir. Böyle durumlarda problem kısmi gözlemlenebilirlik olarak yeniden formüle edilebilir.

4.4 Kısmi Gözlemlerle Arama

Partial observability'de ajanın percept'leri tam fiziksel state'i belirlemeye yetmez. Ajan birden fazla fiziksel state'ten birinde olabileceğini düşündüğünde temel kavram belief state'tir: şimdiye kadar alınan percept ve yapılan action dizisi göz önüne alındığında mümkün olan fiziksel state'lerin kümesi.

Belief state $b = \{\text{ajanın içinde bulunması mümkün fiziksel state'ler}\}$

4.4.1 Gözlem Olmadan Arama: Sensorless / Conformant Problem

Percept hiçbir bilgi vermiyorsa problem sensorless veya conformant problem olarak adlandırılır. Ajan hangi fiziksel state'te olduğunu bilmeseyse bile bazı sensorless problemler çözülebilir. Üretim sistemlerinde pahalı veya güvenilir sensörler kullanmadan parçaları bilinmeyen başlangıç yönlerinden belirli konuma zorlayan eylem dizileri buna örnektir.

Sensorless vacuum world'de ajan haritayı bilir ama konumunu ve kir dağılımını bilmez. Başlangıç belief state'i tüm sekiz fiziksel state olabilir: $\{1,2,3,4,5,6,7,8\}$. Right yapılırsa süpürge kesin olarak sağ tarafta olacağından belief state $\{2,4,6,8\}$ 'e düşer. [Right,Suck] sonrası $\{4,8\}$, [Right,Suck,Left,Suck] sonrası ise hangi başlangıç state'i olursa olsun goal state 7 elde edilir. Buna dünyayı belirli bir state'e coerce etmek denir.

$\{1,2,3,4,5,6,7,8\} \xrightarrow{\text{Right}} \{2,4,6,8\} \xrightarrow{\text{Suck}} \{4,8\} \dots \xrightarrow{} \{7\}$

Sensorless problem fiziksel state space yerine belief-state space'de çözülür. Ajan fiziksel state'ini bilmeseyse de kendi belief state'ini bildiği için bu yeni uzay ajan açısından tam gözlemlenebilir. Fiziksel problemde N state varsa teorik olarak 2^N belief state bulunabilir; ancak bunların çoğu başlangıçtan erişilemeyebilir.

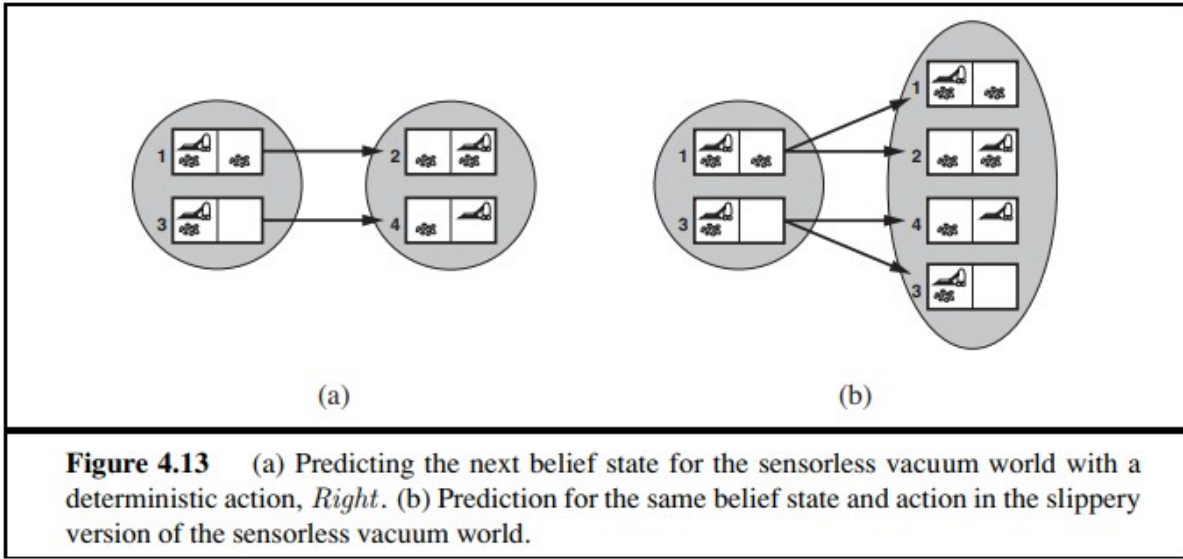
Belief-State Probleminin Kurulması

- Belief state'ler: fiziksel state kümesinin tüm olası alt kümeleri.
- Başlangıç: çoğunlukla tüm fiziksel state'ler kümesi; ön bilgi varsa daha küçük olabilir.
- Actions: illegal action zararsızsa mevcut fiziksel state'lerdeki action'ların birleşimi; tehlikeliyse tüm state'lerde yasal action'ların kesişimi alınabilir.
- Transition: belief state içindeki her fiziksel state'e action uygulanır ve tüm olası sonuçlar birleştirilir.
- Goal test: belief state içindeki bütün fiziksel state'ler hedef olmalıdır.
- Path cost: burada action maliyetinin belief state içindeki fiziksel state'lerde aynı olduğu varsayılır.

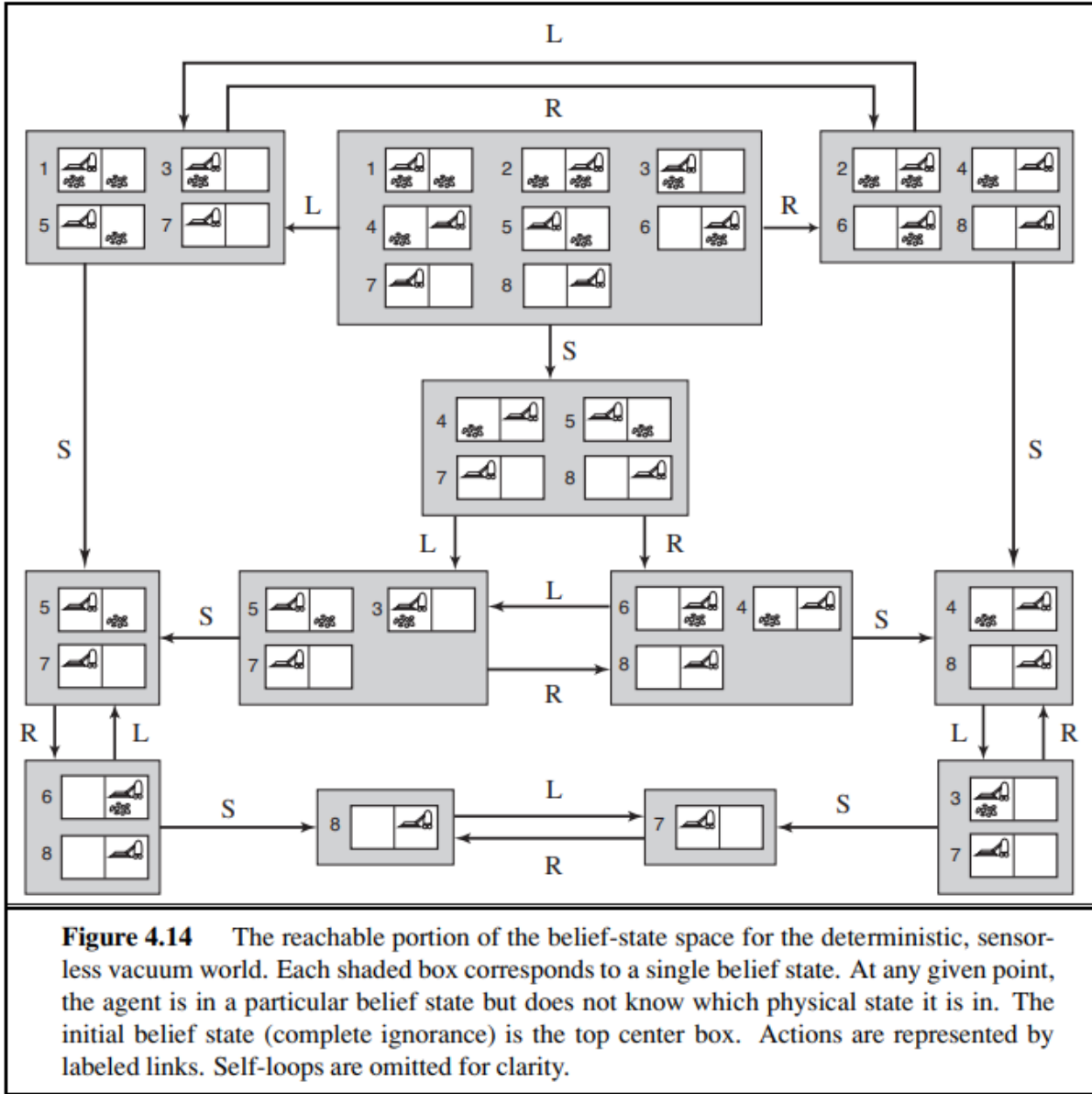
Deterministik: $\text{RESULT}(b,a) = \{ \text{RESULT_P}(s,a) : s \in b \}$

Nondeterministik: $\text{RESULT}(b,a) = \bigcup_{s \in b} \text{RESULTS_P}(s,a)$

Action sonrası yeni belief state üretme işlemine prediction denir ve $\hat{b} = \text{PREDICT}(b,a)$ biçiminde gösterilir.



Şekil 4.13 - (a) Deterministik sensorless vacuum world'de $\{1,3\}$ belief state'i *Right* ile $\{2,4\}$ olur. (b) Slippery sürümde aynı action olası sonuçları genişleterek $\{1,2,3,4\}$ üretir.



Şekil 4.14 - Deterministik sensorless vacuum world için erişilebilir belief-state uzayı. $2^8=256$ teorik belief state'e karşılık yalnızca 12 tanesi erişilebilirdir.

Belief state'lerde subset-superset ilişkileri ek pruning fırsatı verir. Bir action sequence b belief state'inin bütün fiziksel state'lerinde çalışıyorsa, b'nin herhangi bir alt kümesinde de çalışır. Bu nedenle çözülmüş daha büyük bir belief state, alt kümelerin çözülebilirliğini garanti eder; çözülemez küçük bir subset ise onu içeren supersetlerin de çözülemez olduğunu gösterebilir.

Pratikte asıl güçlük yalnızca 2^N belief state sayısı değildir; tek bir belief state'in kendisi astronomik sayıda fiziksel state içerebilir. Örneğin 10x10 vacuum world için başlangıç belief state'i yaklaşık 100×2^{100} fiziksel state içerebilir. Bu nedenle açık liste yerine mantıksal/kompakt gösterimler veya incremental belief-state search gerekir.

4.4.2 Gözlemlerle Arama

Genel partial observable problemde ortamın percept üretme biçimi de tanımlanmalıdır. Local-sensing vacuum world'de ajan konumunu ve bulunduğu karenin kir durumunu algılar, fakat diğer karenin kirini algılayamaz. PERCEPT(s) işlevi state s'de alınan percept'i döndürür. Fully observable problemde PERCEPT(s)=s, sensorless problemde PERCEPT(s)=null özel durumları elde edilir.

Aynı percept birden fazla fiziksel state tarafından üretilebilir. Örneğin [A,Dirty], state 1 ve 3'te mümkündür; dolayısıyla ilk percept bu ise başlangıç belief state {1,3} olur.

Bir action'dan sonra belief state geçişi üç aşamada ele alınır:

1. Prediction: $b\text{-hat} = \text{PREDICT}(b,a)$. Action'ın fiziksel sonuçları hesaplanır.
2. Observation prediction: $b\text{-hat}$ içindeki state'lerin üretebileceği tüm percept'ler bulunur.
3. Update: Gerçek percept o ile uyumlu state'ler tutulur, diğerleri elenir.

$$\text{POSSIBLE-PERCEPTS}(b\text{-hat}) = \{ \text{PERCEPT}(s) : s \in b\text{-hat} \}$$

$$\text{UPDATE}(b\text{-hat},o) = \{ s \in b\text{-hat} : \text{PERCEPT}(s)=o \}$$

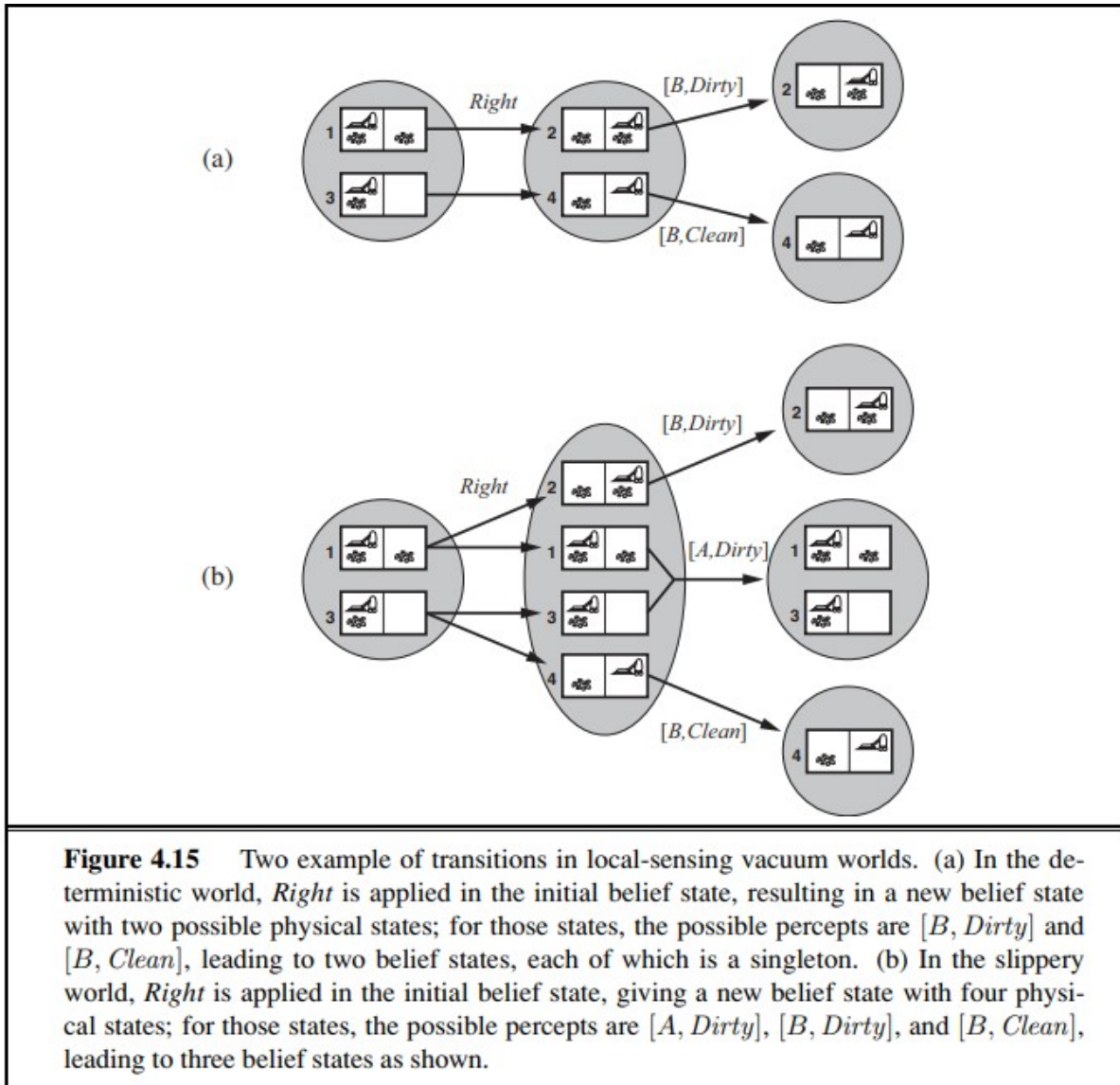


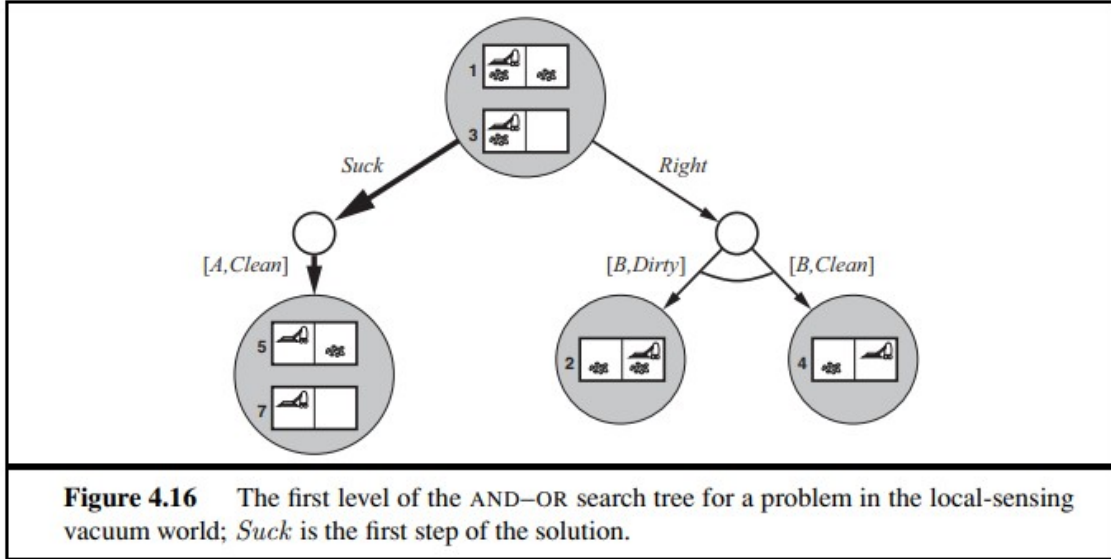
Figure 4.15 Two example of transitions in local-sensing vacuum worlds. (a) In the deterministic world, *Right* is applied in the initial belief state, resulting in a new belief state with two possible physical states; for those states, the possible percepts are [B, Dirty] and [B, Clean], leading to two belief states, each of which is a singleton. (b) In the slippery world, *Right* is applied in the initial belief state, giving a new belief state with four physical states; for those states, the possible percepts are [A, Dirty], [B, Dirty], and [B, Clean], leading to three belief states as shown.

Şekil 4.15 - Local-sensing vacuum world'de iki belief-state geçişi. (a) Deterministik dünyada *Right* sonrası {2,4}; percept belief state'i {2} veya {4} singleton'ına indirir. (b) Slippery dünyada prediction {1,2,3,4}'e genişler, ardından percept'e göre üç farklı belief state oluşur.

Temel ritim: Nondeterminism prediction aşamasında belirsizliği büyütebilir; observation/update ise bilgi sağlayarak belirsizliği küçültür.

4.4.3 Kısmi Gözlemlenebilir Problemleri Çözme

RESULTS işlevi belief-state seviyesinde kurulduğunda Figure 4.11'deki AND-OR arama doğrudan uygulanabilir. Ancak artık ağacın düğümleri fiziksel state değil belief state'lerdir. Çözüm, gerçek state'i test eden değil, percept veya belief state'e göre dallanan bir conditional plan olmalıdır.



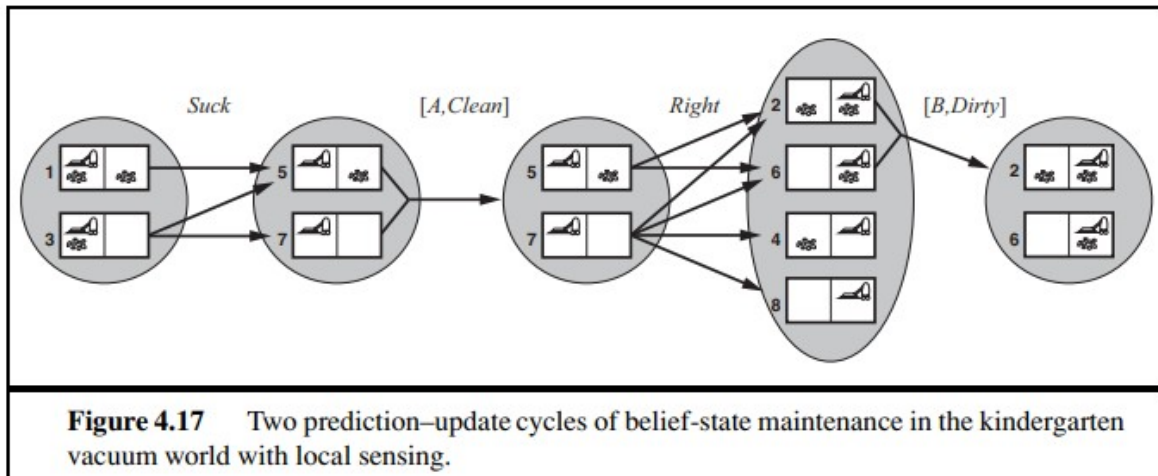
Şekil 4.16 - Local-sensing vacuum world'de AND-OR aramasının ilk seviyesi. İlk percept [A,Dirty] olduğunda başlangıç belief state {1,3}; çözümün ilk adımı Suck'tır.

Örnek çözüm: [Suck, Right, if Bstate={6} then Suck else []]. Koşulun "State=6" değil "Belief state={6}" olması önemlidir; partial observable ajan gerçek fiziksel state'i doğrudan test edemez.

4.4.4 Kısmi Gözlemlenebilir Ortamlar İçin Ajan

Bu ajan, klasik problem-solving agent'e benzer biçimde problem kurar, arama algoritması çağırır ve bulunan çözümü yürütür. İki fark vardır: çözüm bir action sequence değil conditional plan olabilir ve ajan her action/percept sonrasında belief state'ini güncel tutmak zorundadır.

$$b_new = \text{UPDATE}(\text{PREDICT}(b_old, a), o)$$

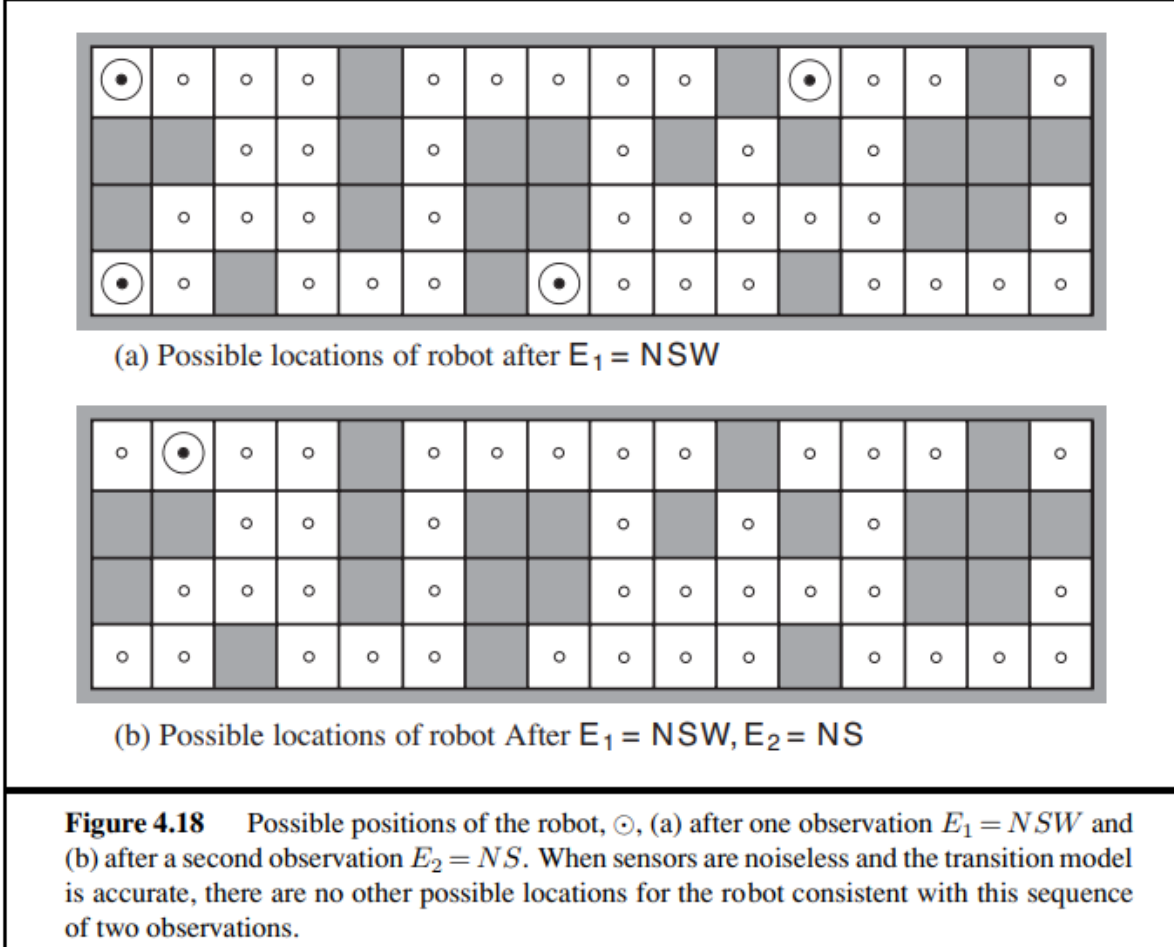


Şekil 4.17 - Kindergarten vacuum world'de iki prediction-update döngüsü. Action olası state kümesini genişletebilir, gelen percept uyumsuz state'leri eleyerek belief state'i tekrar daraltır.

Belief state'i sürekli güncel tutma işlevi monitoring, filtering veya state estimation adlarıyla anılır. Yukarıdaki formül recursive state estimator'dır; yeni belief state'i tüm geçmiş percept dizisini yeniden işlemek yerine önceki belief state'ten hesaplar. Gerçek zamanlı sistemlerde bu hesabın percept'lerin geliş hızına yetişmesi gerekir.

Localization Örneği

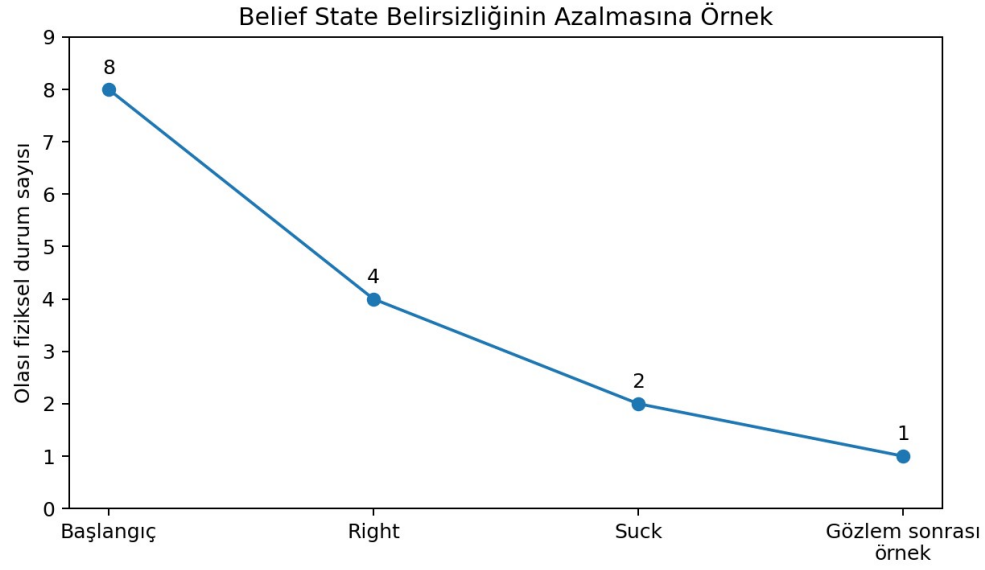
Robot localization probleminde robot haritayı bilir fakat kendi konumunu bilmez. Dört sonar sensörü kuzey, güney, doğu ve batıda engel olup olmadığını hatasız söyler. Ancak hareket sistemi arızalıdır: Move action'ı robotu komşu karelerden rastgele birine götürür. Amaç percept ve action dizilerinden mevcut konumu belirlemektir.



Şekil 4.18 - İlk percept $E_1=NSW$ sonrası robot için dört olası konum kalır. Bir nondeterministic Move ve ikinci percept $E_2=NS$ sonrası yalnızca tek konum bütün gözlemlerle tutarlı kalır.

İlk observation NSW kuzey, güney ve batıda engel bulunduğunu gösterir ve belief state'i dört kareye indirir. Move action'ı nondeterministic olduğundan prediction bu dört konumun bir adım komşularını kapsayacak biçimde belief state'i genişletir. İkinci observation NS geldiğinde yalnızca tek konum tutarlı kalır.

UPDATE(PREDICT(UPDATE(b, NSW), Move), NS) = tek fiziksel konum



Grafik - Belief state'in action ve observation'larla küçülmesine ilişkin öğretici bir örnek. Gerçek sayılar probleme göre değişir; amaç "prediction genişletebilir, update daraltır" fikrini göstermektir.

Bazı ortamlarda percept yeterince ayırt edici değildir. Uzun doğu-batı koridorunda robot uzun süre yalnızca NS gözlemi alabilir ve koridorun tam olarak neresinde olduğunu belirleyemeyebilir. Sensörün bilgi değeri, farklı state'leri birbirinden ayırabilmesine bağlıdır.

4.5 Çevrimiçi Arama Ajanları ve Bilinmeyen Ortamlar

Offline search algoritmaları önce tam çözümü hesaplar, sonra gerçek dünyada uygular. Online search ajanı ise hesaplama ve eylemi iç içe geçirir: hareket eder, ortamı gözlemler ve bir sonraki hareketi hesaplar. Dinamik veya yarı dinamik alanlarda uzun süre hesap yapmanın maliyeti olduğu için bu yaklaşım değerlidir. Nondeterministic ortamlarda da yalnızca gerçekten ortaya çıkan contingency'lere hesaplama ayırmayı sağlar.

OFFLINE ARAMA

Problemi tamamen çöz → Tam planı üret → Planı uygula

ONLINE ARAMA

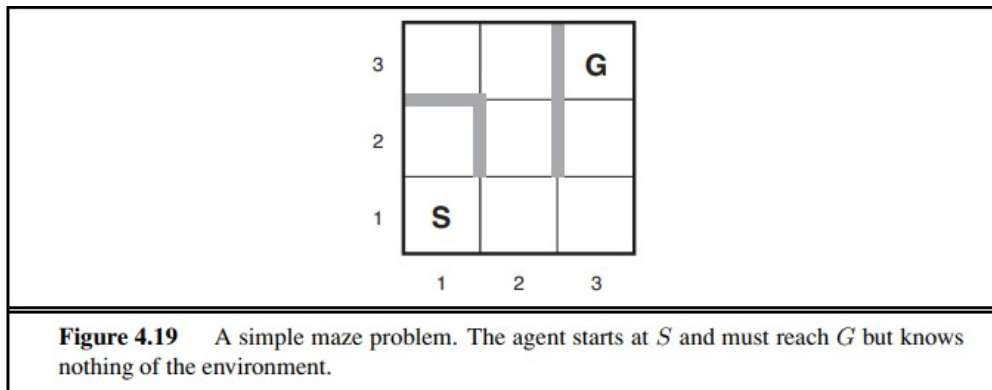
Hareket et → Gözlemle → Öğren/Hesapla → Sonraki hareket

Şema - Offline ve online arama arasındaki temel iş akışı farkı.

Online search özellikle unknown environment için zorunludur. Ajan hangi state'lerin var olduğunu veya action'ların ne yaptığını bilmediğinde bir exploration problem ile karşılaşır ve action'larını deney olarak kullanarak çevreyi öğrenir. Yeni bir binaya bırakılan haritasız robot bunun klasik örneğidir; bir bebeğin eylemlerinin sonuçlarını yavaş yavaş keşfetmesi de benzer bir süreçtir.

4.5.1 Online Search Problemleri

Burada ortamın deterministik ve fully observable olduğunu varsayalım, fakat ajan yalnızca ACTIONS(s), step-cost $c(s,a,s')$ ve GOAL-TEST(s) bilgilerini bilir. En önemli eksik RESULT(s,a)'dır: ajan bir action'ın sonucunu ancak gerçekten state s'de bulunup action'ı uyguladıktan sonra öğrenebilir.



Şekil 4.19 - Basit bilinmeyen labirent. Ajan S'den G'ye ulaşmalıdır fakat iç duvarların ve geçişlerin yapısını önceden bilmez.

Ajan hedefin yerini biliyorsa admissible heuristic kullanabilir; örneğin grid üzerinde Manhattan distance hedefe yaklaşık uzaklık sağlayabilir. Online aramada performansın gerçek maliyeti, ajanın gerçekten yürüdüğü toplam yoldur. Bu maliyet, çevreyi önceden bilen optimal ajanın maliyetiyle karşılaştırılarak competitive ratio elde edilir.

Competitive ratio = online ajanın gerçek maliyeti / önceden bilen optimal maliyet

Irreversible Actions, Dead Ends ve Adversary Argument

Bazı action'lar geri döndürülemez olabilir ve ajan yanlış seçimle hedefe artık ulaşılamayan dead end'e girebilir. Hiçbir online algoritmanın bütün state space'lerde dead end'den kaçmayı garanti edemeyeceği Figure 4.20(a)'daki adversary argument ile gösterilir: S ve A'ya kadar iki dünya aynı görünür, fakat A'daki doğru seçim farklıdır. Ajan aynı bilgiye sahip olduğu için aynı kararı vermek zorundadır ve dünyalardan en az birinde başarısız olur.

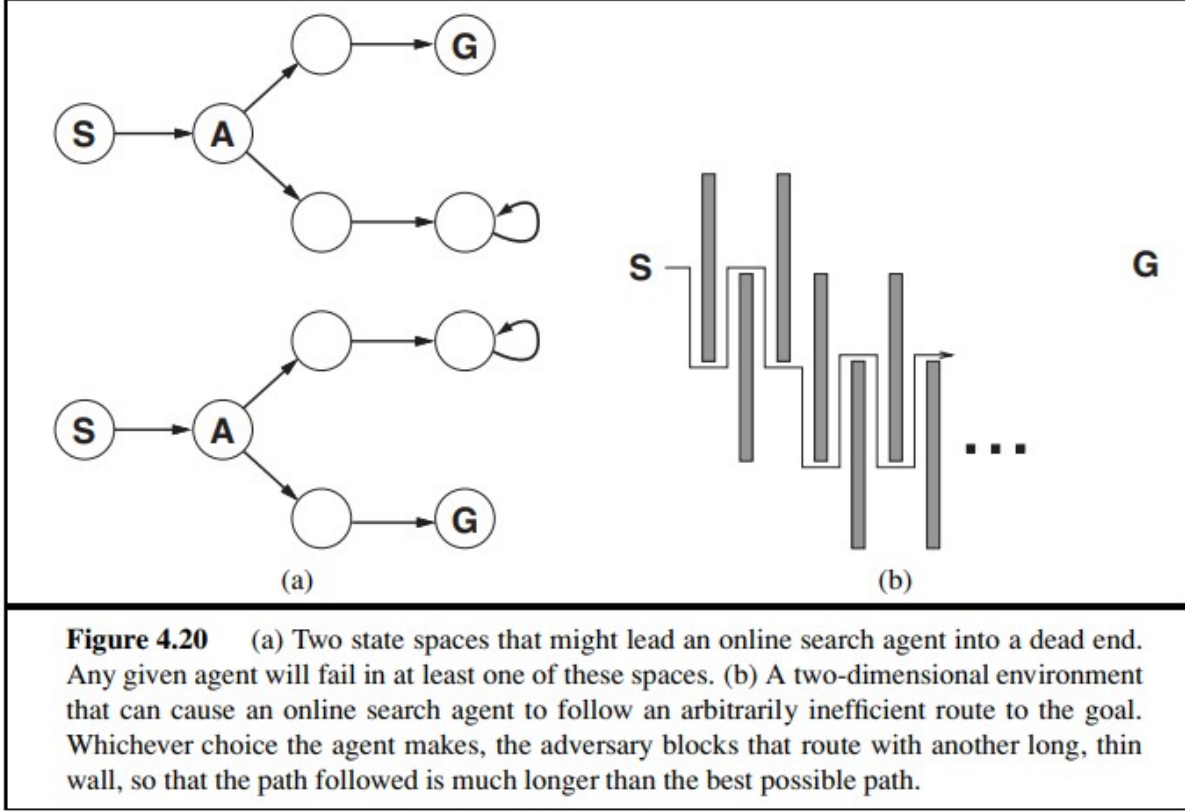


Figure 4.20 (a) Two state spaces that might lead an online search agent into a dead end. Any given agent will fail in at least one of these spaces. (b) A two-dimensional environment that can cause an online search agent to follow an arbitrarily inefficient route to the goal. Whichever choice the agent makes, the adversary blocks that route with another long, thin wall, so that the path followed is much longer than the best possible path.

Şekil 4.20 - (a) Online ajanın ayırt edemediği iki ortamda hangi dalı seçerse seçsin en az birinde dead end'e düşmesi mümkündür. (b) Çıkmaz olmasa bile rakip, ajanın seçtiği yönü uzun duvarlarla tekrar tekrar kapatarak rotayı keyfi derecede verimsiz yapabilir.

Analizi mümkün hale getirmek için çoğu zaman safely explorable state space varsayılır: erişilebilir her state'ten en az bir goal state'e ulaşılabilmelidir. Reversible action'lara sahip labirent ve 8-puzzle gibi yönsüz grafikler güvenli biçimde keşfedilebilir. Buna rağmen yolların maliyeti sınırsızsa bounded competitive ratio yine garanti edilemez.

4.5.2 Online Search Ajanları ve Online DFS

Online ajan her action'dan sonra hangi state'e ulaştığını algılar ve bu bilgiyi çevre haritasına ekler. Offline A* uzayın bir yerindeki düğümü genişletip hemen başka bölgedeki düğüme geçebilir çünkü action'lar simüle edilir. Online ajan ise yalnızca fiziksel olarak bulunduğu state'in successor'larını keşfedebilir. Bu nedenle yerel genişletme sırası avantajlıdır; depth-first search bu özelliğe sahiptir.

```
function ONLINE-DFS-AGENT(s_new) returns an action
  kalıcı: result, untried, unbacktracked, önceki s ve a
  if GOAL-TEST(s_new) then return stop
  if s_new yeni state ise untried[s_new] <- ACTIONS(s_new)
  if önceki s varsa
    result[s,a] <- s_new
    s'yi unbacktracked[s_new] listesine ekle
  if untried[s_new] boşsa
```

```

if unbacktracked[s_new] boşsa return stop
else önceki state'e götüren geri action'ı seç
else
  a <- henüz denenmemiş bir action
  s <- s_new
  return a

```

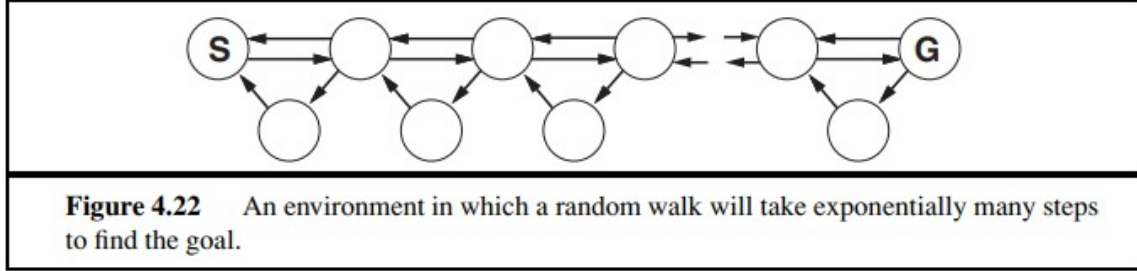
Şekil 4.21 - Online DFS ajanının sadeleştirilmiş sözde kodu.

Offline DFS'te backtracking bellekte stack üzerinde yapılır; online DFS'te ajan fiziksel olarak geri yürümek zorundadır. Bu nedenle Figure 4.21'deki algoritma her action'ın başka bir action ile geri alınabildiği state space'lerde geçerlidir. En kötü durumda her kenar ileri ve geri olmak üzere tam iki kez geçilir; tüm ortamı keşfetme açısından bu optimaldir, fakat sadece yakın bir goal bulmak için çok verimsiz olabilir.

4.5.3 Online Yerel Arama

Hill climbing yalnızca mevcut state'i bellekte tuttuğu ve yerel komşulara geçtiği için doğal olarak online niteliktedir. Ancak local maksimum/minimumda takılabilir. Offline random restart gerçek bir robot için uygulanamaz; ajan kendisini rastgele başka bir state'e ıslınlayamaz.

Random walk mevcut action'lardan rastgele birini seçer. Sonlu state space'de uygun koşullarda sonunda goal bulabilir veya tüm ortamı keşfedebilir, fakat çok yavaş olabilir.

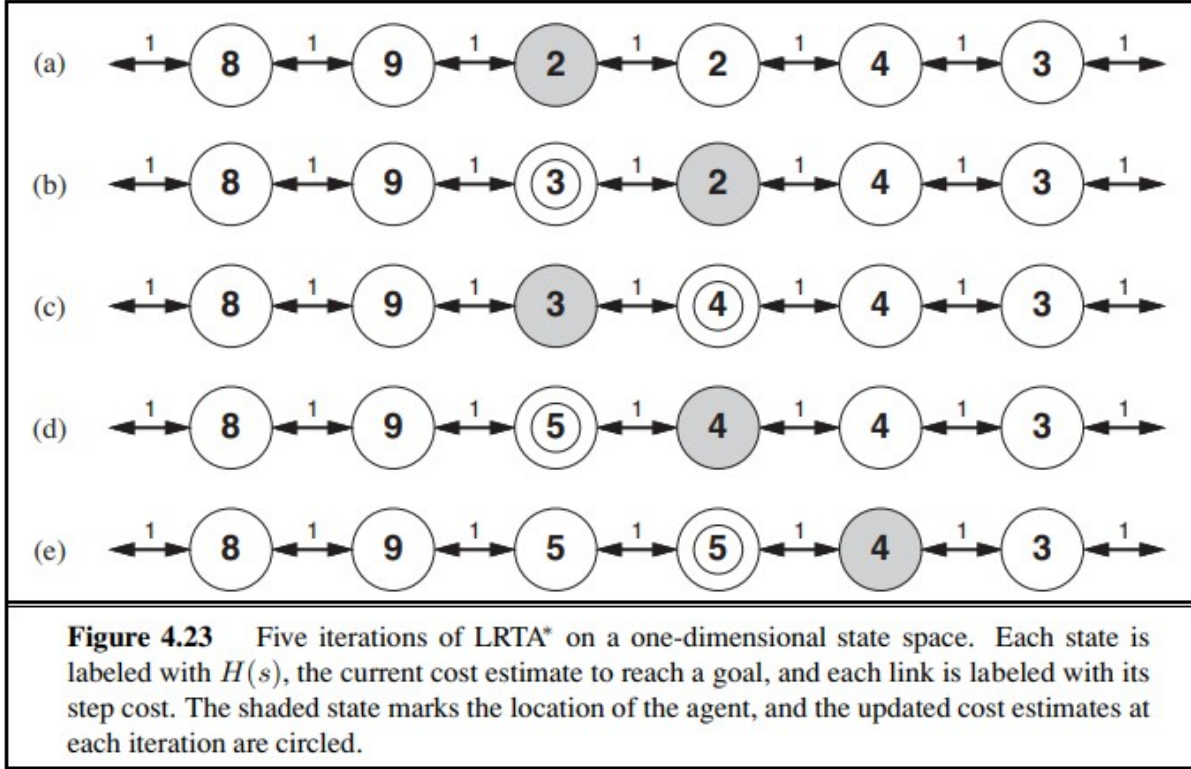


Şekil 4.22 - Random walk'ın goal'a ulaşmak için üstel sayıda adım gerektirebildiği yapı. Her aşamada geri ilerleme olasılığı ileri ilerleme olasılığından daha yüksektir.

Randomness yerine memory eklemek daha etkili olabilir. Her ziyaret edilen state için goal'a kalan mevcut en iyi maliyet tahmini $H(s)$ saklanır. Başlangıçta $H(s)=h(s)$ 'dir; ajan deneyim kazandıkça tahminleri günceller.

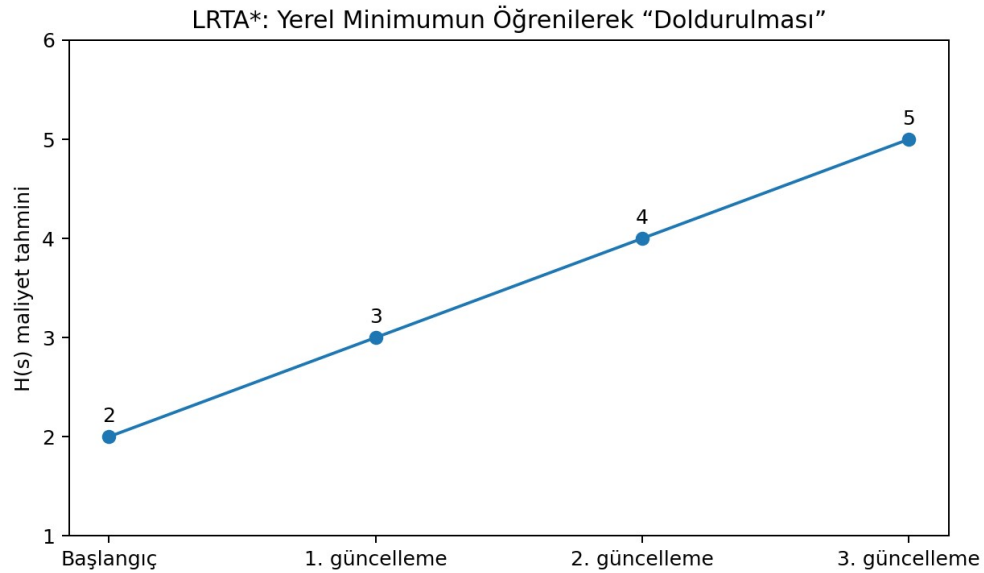
$$\text{Bir komşu üzerinden tahmini toplam maliyet} = c(s,a,s') + H(s')$$

LRTA*: Learning Real-Time A*



Şekil 4.23 - Bir boyutlu state space üzerinde LRTA*'ın beş iterasyonu. Daire içindeki sayılar $H(s)$, kenarlardaki sayılar step cost'tur. Gri state ajanın konumu, çift daire güncellenen değeri gösterir.

İlk durumda ajan $H=2$ olan gri state'tedir. Sol komşu üzerinden maliyet $1+9=10$, sağ komşu üzerinden $1+2=3$ olduğundan sağa gider. Bu deneyim mevcut state'in $H=2$ tahmininin aşırı iyimser olduğunu gösterir; en iyi hareket bile en az 3 maliyet gerektirdiği için H değeri 3'e yükseltilir. Ajan bir süre ileri geri hareket ettikçe $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ biçiminde local minimumu adeta doldurur ve sonunda sağa kaçabilecek hale gelir.



Grafik - LRTA*'ta aşırı iyimser bir $H(s)$ tahmininin deneyimle artmasına yönelik sadeleştirilmiş gösterim.

```
function LRTA*-AGENT( $s_{new}$ ) returns an action
  kalıcı: result, H, önceki s ve a
```

```

if GOAL-TEST(s_new) then return stop
if s_new yeni ise H[s_new] <- h(s_new)
if önceki s varsa
    result[s,a] <- s_new
    H[s] <- min_b LRTA*-COST(s,b,result[s,b],H)
a <- ACTIONS(s_new) içinde LRTA*-COST'u minimize eden action
s <- s_new
return a

function LRTA*-COST(s,a,s_new,H)
    if s_new bilinmiyorsa return h(s)
    else return c(s,a,s_new) + H[s_new]

```

Şekil 4.24 - LRTA* ajanının Türkçeleştirilmiş sözde kodu.

Henüz denenmemiş action'ın hedefe en düşük olası maliyetle ulaşabileceği varsayılır. Bu optimism under uncertainty yaklaşımı, ajanın bilinmeyen fakat umut verici yolları keşfetmesini sağlar. Sonlu ve safely explorable ortamda LRTA* goal bulmayı garanti eder; sonsuz state space'de complete değildir. n state'li ortamı en kötü durumda $O(n^2)$ adımda keşfedebilir, pratikte çoğu zaman daha iyidir.

4.5.4 Online Aramada Öğrenme

Online search ajanının başlangıç bilgisizliği öğrenme fırsatı yaratır. Birincisi, ajan her deneyimden $RESULT[s,a]=s'$ ilişkisini kaydederek çevrenin haritasını öğrenir. Ortam deterministik olduğu için her state-action çifti için tek deneyim yeterlidir. İkincisi, LRTA* gibi yerel yöntemler $H(s)$ maliyet tahminlerini giderek daha doğru hale getirir.

Uygun keşif koşullarında bu güncellemeler gerçek optimal maliyetlere yakınsar. Eğer her state'in gerçek goal maliyeti biliniyorsa, en düşük maliyetli successor'a gitmek, yani saf hill climbing, optimal hale gelir. Sorun hill climbing'in yerel karar vermesi değil; başlangıçtaki maliyet tahminlerinin eksik veya yanlış olmasıdır.

Online DFS yalnızca tek tek state-action sonuçlarını ezberler; örneğin Up'ın (1,1)'den (1,2)'ye götürdüğünü öğrendikten sonra otomatik olarak Down'ın geri döndüreceğini veya Up'ın başka sütunlarda y koordinatını artırdığını çıkarmaz. Daha güçlü öğrenme için ajanın “duvar yoksa Up y'yi 1 artırır” gibi genel kuralları temsil edebilmesi ve özel gözlemlerden bu kuralları öğrenebilmesi gerekir. Bu fikir kitabın sonraki temsil ve öğrenme bölümlerine geçiş sağlar.

4.6 Ünite Özeti

Ünite 4, klasik arama varsayımlarını adım adım gevşetir. Önce yolun önemsiz olduğu optimizasyon problemlerinde yerel aramaya geçilir; sonra sürekli uzaylar, nondeterministic action'lar, partial observability ve en sonunda başlangıçta bilinmeyen ortamlar ele alınır.

Durum	Klasik çözüm biçimi	Gerekli temel fikir
Yol önemsiz, amaç en iyi state	Optimum state	Local search / optimization
Local optimum riski	Tek state araması	Simulated annealing / restart / memory
Bir action birden fazla sonuç	Conditional plan	AND-OR search
Gerçek state tam görülemiyor	Belief-state conditional plan	Predict + Update
Çevre başlangıçta bilinmiyor	Eylem ve arama iç içe	Online search / exploration / learning

Tek cümlelik bölüm özeti: Klasik aramada “haritayı biliyorum, hangi yolu izlemeliyim?” sorusu sorulurken Ünite 4 ilerledikçe soru “hangi durumun en iyi olduğunu nasıl bulurum, action’ın sonucu kesin değilse ne yaparım, state’i göremiyorsam neye inanmalıyım ve haritayı bile bilmiyorsam hareket ederken nasıl öğrenirim?” biçimine dönüşür.

ALİŞTIRMALAR 4.1-4.14

Aşağıdaki kısımda kullanıcı tarafından sağlanan alıştırmalar Türkçeye çevrilmiş ve her biri için çözüm/çözüm yaklaşımı açıklanmıştır. Programlama gerektiren sorularda, ilgili önceki bölüm haritaları veya veri setleri bu belgeye dahil olmadığı için sahte deney sonucu verilmemiş; doğru deney tasarımı ve beklenen davranış açıklanmıştır.

4.1 Özel Durumlardan Hangi Algoritma Çıkar?

Soru: Aşağıdaki özel durumların her biri hangi algoritmaya karşılık gelir? (a) Local beam search $k=1$. (b) Tek başlangıç state'i ve tutulan state sayısında sınır yok. (c) Simulated annealing $T=0$. (d) Simulated annealing $T=\infty$. (e) Genetic algorithm population $N=1$.

Madde	Sonuç	Neden
a	Hill climbing	Tek state tutulur ve en iyi komşuya geçilir.
b	Breadth-first search	Bir seviyedeki tüm ardılar tutulur; state sayısı sınırı yoktur.
c	First-choice / stochastic hill climbing	Kötü hamlenin kabul olasılığı sıfıra gider.
d	Random walk	Her rastgele seçilen hamle kabul edilir.
e	Mutation tabanlı random walk	Tek birey kendisiyle crossover yapar; değişim mutation'dan gelir.

4.2 Demiryolu Parçalarını Simulated Annealing ile Yerleştirme

Soru: Ray parçaları tam oturmuyor ve doğru hizadan her iki yana en fazla 10° dönebiliyor. Problemi simulated annealing ile çözülecek şekilde formüle edin.

Çözüm: Her bağlantının sapma açısını θ_i ile gösterelim ve $-10^\circ \leq \theta_i \leq 10^\circ$ olsun. State $s=(\theta_1, \dots, \theta_n)$ ile temsil edilir; parça sırası değişebiliyorsa sıra da state'e eklenir. Amaç fonksiyonu, yolun hedef noktaya olan karesel uzaklığı, son yön hatası ve çakışma cezalarının ağırlıklı toplamı olabilir.

$$E(s) = \lambda_1 \cdot d(\text{son}, \text{hedef})^2 + \lambda_2 \cdot (\text{yön hatası})^2 + \lambda_3 \cdot \text{çakışma cezası}$$

Neighbor üretmek için rastgele bir bağlantı seçilip θ_i küçük δ miktarı değiştirilir. Daha düşük enerji kesin kabul edilir; daha yüksek enerji $\exp(-(E_{\text{new}} - E_{\text{old}})/T)$ olasılığıyla kabul edilir. Böylece algoritma çok sayıdaki local minimumdan kaçabilir.

4.3 TSP için Hill Climbing ve Genetic Algorithm

Soru (a): TSP'leri hill climbing ile çözün ve MST heuristic kullanan A* ile elde edilen optimal çözümlerle karşılaştırın. (b): Aynı deneyi genetic algorithm ile tekrarlayın.

Hill climbing'de state şehir permütasyonudur. Amaç tur uzunluğunu minimize etmektir. Komşuluk için 2-opt uygundur: turdaki iki kenar kesilir ve aradaki bölüm ters çevrilir. En çok iyileştiren komşu seçilir. A* + admissible MST heuristic optimal referans sağlar; hill climbing çok daha az bellek kullanır fakat local optimum nedeniyle $\text{Cost}_{\text{HC}} \geq \text{Cost}_{\text{optimal}}$ olabilir.

Genetic algorithm'da birey yine şehir permütasyonudur. Fitness = $1/\text{TourLength}$ veya $C/\text{TourLength}$ seçilebilir. Normal crossover şehirleri tekrarlayabileceği için Ordered Crossover, PMX veya Cycle Crossover gibi permutation-preserving operatörler gerekir. Mutation iki şehri takas etmek veya bir alt diziyi ters çevirmek olabilir. GA local optimumdan kaçmada daha güçlü olabilir fakat optimalite garantisi vermez.

4.4 8-Puzzle ve 8-Queens Deneyi

Soru: Çok sayıda örneği steepest-ascent, first-choice, random-restart hill climbing ve simulated annealing ile çözün; arama maliyeti ve çözüm yüzdesini optimal maliyete göre grafikleyin.

8-vezir için standart steepest-ascent'in kitapta verilen yaklaşık başarı oranı %14'tür; 100 sideways move sınırıyla yaklaşık %94'e çıkar. First-choice bir adımda daha az successor değerlendirir fakat daha fazla adım gerekebilir. Random-restart local optimum sorununu tekrarlı denemelerle azaltır. Simulated annealing'in sonucu soğutma schedule'ına çok bağlıdır: hızlı soğutma hill climbing'e yaklaşıp, yavaş soğutma daha iyi keşif yapar ama maliyeti yükseltir.

8-puzzle için önemli nokta, çözümde başlangıç state'inden goal'a gerçek yasal hareket dizisinin önemli olmasıdır. Bu nedenle "başka rastgele bir board'dan yeniden başlamak" verilen başlangıç örneğinin çözümü sayılmaz. Deney raporunda her yöntem için çözüm yüzdesi, ortalama expanded/evaluated successor sayısı, gerçek hareket sayısı ve optimal maliyete oran raporlanmalıdır.

4.5 AND-OR Search'te Global Repeated-State Kontrolü

Soru: Algoritma yalnızca mevcut root-to-node path üzerinde tekrar kontrol ediyor. Tüm ziyaret edilen state'ler saklanırsa hangi bilgi tutulmalı ve repeated state bulunduğu nasıl kullanılmalıdır?

Sadece visited true/false yeterli değildir. En az SOLVED, FAILED ve IN_PROGRESS durumları tutulmalı; SOLVED state için bulunan subplan veya onun label'ı saklanmalıdır. Bir state daha önce SOLVED ise alt plan tekrar aranmaz, label'a referans verilir. Güvenli FAILED memoization için failure'ın path/ancestor bağlamına bağlı olup olmadığına dikkat edilmelidir; bir state yalnızca mevcut ancestor'a geri dönmek zorunda kaldığı için başarısız olduysa başka bağlamda çözülebilir. IN_PROGRESS tekrarları acyclic çözüm ararken cycle olarak işaretlenir.

4.6 AND-OR-GRAPH-SEARCH'ü Döngüsel Plan Üretecek Şekilde Değiştirme

Plan sonucu (plan, tür) olarak tutulabilir; tür ACYCLIC, CYCLIC veya FAILURE'dir. Her state'e bir label verilir. OR-SEARCH mevcut path'teki state'e dönerse doğrudan failure yerine o label'a geri dönüş içeren cyclic aday üretir; ancak başka action'larda acyclic çözüm bulunup bulunmadığını araştırmaya devam eder ve mümkünse acyclic planı tercih eder. AND-SEARCH bütün outcomes için çözüm gerektirir; herhangi bir dal cyclic ise toplam plan cyclic işaretlenir. Sonsuza kadar hedefe çıkışı olmayan salt self-loop çözüm kabul edilmez.

Slippery vacuum world için örnek: Suck; L1: Right; eğer State=5 ise L1'e dön, değilse Suck.

Slippery+erratic sürümde ilk Suck state 7'ye doğrudan götürebilir; state 5'e götürürse Right başarılı olana kadar döngü kurulur ve sonra kalan kir temizlenir.

4.7 Sensorless Problem İçin Admissible Heuristic

Soru: Belief state b içindeki her fiziksel state s için fully observable problemde gerçek optimal maliyet $h^*(s)$ biliniyorsa sensorless problem için admissible $h(b)$ bulun ve ispatlayın.

$$h(b) = \max_{\{s \in b\}} h^*(s)$$

İspat: Sensorless plan aynı action sequence ile b içindeki her fiziksel state'i çözmek zorundadır. Dolayısıyla gerçek sensorless optimal maliyet $h^*(b)$, her bir $h^*(s)$ 'den küçük olamaz. Bu nedenle $\max h^*(s) \leq h^*(b)$ ve heuristic asla gerçek maliyeti aşmaz; admissible'dir. Sensorless vacuum başlangıcında fiziksel state'ler arasındaki en büyük tek-state optimal maliyet 3 iken, garantili sensorless planın maliyeti 4 olabilir; heuristic güçlü fakat kusursuz değildir. Erişilebilir belief state sayısı küçük olduğundan A* bu örnekte iyi performans gösterir.

4.8 Belief State Subset-Superset İlişkileri

- (a) π action sequence'i b belief state'indeki her state'i goal'a götürüyorsa, b'nin herhangi bir alt kümesindeki state'ler de zaten b içinde olduğundan π o alt küme için de çözümdür. Superset için garanti yoktur; eklenen state'ler aynı planla çözülemeyebilir. Tersinden, bir superset çözülebiliyorsa tüm subsetleri çözülebilir; bir subset çözülemiyorsa onu içeren hiçbir superset çözülemez.
- (b) Sensorless graph search'te yeni belief state üretildiğinde yalnızca eşitlik değil subset/superset dominance de kontrol edilir. Maliyetler de hesaba katılmalıdır: daha az belirsiz bir subset daha düşük/eşit g maliyetiyle zaten elde edilmişse daha geniş yeni state budanabilir.
- (c) Partially observable AND-OR search'te başarılı bir plan daha küçük subsetler için yeniden kullanılabilir; çözülemez bir subset onu içeren superset için erken failure sağlar. Bu kontroller AND düğümlerinde her percept outcome'unun çözülmesi zorunluluğuyla birlikte uygulanır.

4.9 Belief State İçinde Action Maliyetleri Farklıysa Optimalite

Tek bir belief state içindeki farklı fiziksel state'lerde aynı action farklı maliyete sahipse "plan maliyeti" artık tek doğal sayıyla tanımlı değildir. Önce karar kriteri seçilmelidir.

- Minimum fiziksel maliyet: iyimser/best-case ölçüdür. Klasik A* scalar maliyetle çalışabilir, fakat garanti maliyetini optimize etmez.
- Maksimum fiziksel maliyet: robust/worst-case yaklaşımın temelidir. Ancak her adımın maksimumunu ayrı ayrı toplamak aşırı muhafazakâr olabilir; doğru kriter tüm mümkün yürütmelerin toplam maliyetinin maksimumudur.
- Maliyet aralığı [min,max]: planlar her zaman tek bir sırada karşılaştırılmaz; ek tercih kriteri gerekir.
- Tüm olası maliyet kümesi: problem set-valued / multiobjective aramaya dönüşebilir ve Pareto-optimal çözümler ortaya çıkabilir.

Expected cost optimize edilecekse olası fiziksel state'ler üzerinde probability distribution gerekir. Bu nedenle "optimal" kavramı kaybolmaz; fakat hangi risk ölçüsüne göre optimal olduğunun açıkça tanımlanması gerekir.

4.10 Sensorless Erratic Vacuum World Neden Çözülemez?

Başlangıç $B_0=\{1,2,3,4,5,6,7,8\}$. Left bunu $BL=\{1,3,5,7\}$, Right ise $BR=\{2,4,6,8\}$ yapar. Erratic Suck, clean state'leri yeniden kirletebildiği ve kirli state'leri birden fazla biçimde temizleyebildiği için bu belief state'leri goal-only kümelere zorlayamaz; uygun union sonuçları $B_0/BL/BR$ gibi kümeleri korur.

Sensorless goal belief state'in bütün elemanları hedef fiziksel state olmalı, yani $b \subseteq \{7,8\}$ gerekir. Erişilebilir belief state'lerden hiçbiri yalnızca 7 ve/veya 8'den oluşmaz. Dolayısıyla ajanın percept olmadan hedefi garanti eden tek bir action sequence'i yoktur; problem unsolvable'dır.

4.11 Navigasyon Ortamının Uygulanması ve Hata Kurtarma

Soru, Exercise 3.7'deki görünür köşelere dayalı robot navigasyonunu bir environment'a dönüştürür. Percept robotun kendisini değil, görünen vertex'lerin robota göre konumlarını verir. Action düz çizgi vektörüdür; engelle kesişirse robot ilk çarpışma noktasında durur. Goal'da sıfır hareket action'ı robotu rastgele başka konuma teleport eder. Her yürünmüş uzaklık bir puan kaybettirir, goal başına 1000 puan kazandırır.

- (a) Agent percept'teki relative vertex düzenini map'te eşleştirerek localization yapar; görünürlük grafiğinde A* veya Dijkstra ile goal'a rota planlar. Teleport sonrası yeniden localization ve replanning gerekir.
- (b) 100 episode için her episode başlangıç/localization, planlanan rota, gerçek yürünmüş mesafe, goal başarısı ve skor loglanmalıdır. Harita/veri verilmeden gerçek sayısal sonuç uydurulmamalıdır.

(c) Action'ların %30'u unintended destination'a gidiyorsa ajan yeni percept ile hata fark eder, mevcut konumu yeniden belirler ve eski planın kaldığı yere dönmek için recovery plan yapar. Geri dönüş action'ı da hata verebileceği için recovery kendi içinde yeniden planlanabilmelidir.

(d) Üç strateji karşılaştırılır: eski konuma dön ve planı sürdür; eski rotadaki en yakın vertex'e bağlan; yeni konumdan goal'a tamamen replan et. Sadece yol mesafesi ölçülürse tam replanning çoğu durumda avantajlı olabilir; search computation maliyeti de skora eklenirse sonuç değişebilir.

(e) Birden fazla konum aynı view'u üretebiliyorsa localization tek state'e inmez; belief state gerekir ve problem partially observable hale gelir. Çözümler percept'e göre dallanan conditional plan/policy biçimini alır.

4.12 Bilinmeyen 3x3 Labirenti Belief-State Olarak Görmek

Ajan başlangıç (1,1), goal (3,3), hareketlerin normal etkilerini bilir; iç duvarları bilmez. 3x3 grid'de yatay komşuluk sınırları $3 \times 2 = 6$, dikey komşuluk sınırları da 6'dır; toplam 12 potansiyel iç duvar vardır. Her biri var/yok olabildiğinden başlangıçta $2^{12} = 4096$ çevre konfigürasyonu mümkündür. Bu 4096 hipotezin alt kümeleri teorik belief state'leri oluşturduğundan çevre-konfigürasyonu kısmı için üst sınır 2^{4096} 'dır.

Başlangıç köşe (1,1)'de Left ve Down dış sınır nedeniyle zaten illegal'dir; yalnızca Up ve Right iç duvara bağlıdır. Bu nedenle dört farklı percept mümkündür: {}, {Up}, {Right}, {Up,Right}.

Tam contingency plan ilk percept'e göre dört dala ayrılır; sonraki state'lerde legal action seti tekrar gözlenir ve plan yeniden dallanır. Naif plan boyutu binlerce çevre hipoteziyle ilişkili olabilir. Ancak goal'un erişilemez olduğu duvar düzenleri de serbestçe izin veriliyorsa hiçbir plan bütün 4096 çevreyi çözemez; "her olası environment için çözüm" ifadesi goal'a erişilebilen admissible labirentlerin kastedildiği varsayımıyla anlamlıdır.

4.13 Robot Navigasyonunda Hill Climbing

(a) Heuristic olarak goal'a Euclidean distance kullanılabilir. Robot her seferinde bu mesafeyi en fazla azaltan hamleyi seçer. Engel nedeniyle goal'a ulaşmak için önce goal'dan uzaklaşmak gerekiyorsa local minimum oluşabilir. Convex engellerde sonuç state/komşuluk temsiline ayrıntısına bağlıdır; genel hill-climbing için garanti yoktur.

(b) U-shaped nonconvex obstacle klasik tuzaktır: robot U'nun içine girip goal'a geometrik olarak çok yaklaşır, fakat çıkışa gitmek için bir süre goal'dan uzaklaşmalıdır. Hill climbing bunu reddeder.

(c) Depth-k sürümünde tüm k adımlık action sequence'leri değerlendirilir, en iyi k-step path seçilir fakat yalnızca ilk action uygulanır; yeni state'te tekrar planlanır. Bu receding-horizon yaklaşımıdır.

(d) Tüm olası ortamlarda çalışan sabit sonlu k yoktur; herhangi bir k için çıkışı k+1 veya daha fazla adım boyunca heuristic'i kötüleştiren daha derin bir tuzak tasarlanabilir.

(e) LRTA* local minimumda H(s) değerlerini deneyimle yükselterek bölgeyi "doldurur". Sonunda kaçış yönündeki c+H değeri daha iyi görünür ve ajan geçici olarak goal'dan uzaklaşmayı kabul ederek tuzaktan çıkar.

4.14 Sonsuz Grid'de Online DFS ve Complete Alternatif

State'ler sonsuz iki boyutlu grid noktaları, action sırası East=(1,0), North=(0,1), West=(-1,0), South=(0,-1) olsun. Online DFS (0,0)'dan ilk East'i dener, sonra her yeni state'te tekrar East henüz denenmemiş olduğundan (1,0),(2,0),(3,0),... şeklinde sonsuza gider. Bu nedenle sonlu derinlikte bulunan (1,-1) state'ine hiçbir zaman ulaşmaz; online DFS sonsuz path içeren reversible state space'de complete değildir.

Ajan mevcut state'in tüm successor state'lerini ve bunlara götüren action'ları görebiliyorsa breadth-first exploration yapılabilir. Keşfedilmemiş frontier state'leri derinlik sırasıyla FIFO yapısında tutulur; bidirected ortamda ajan bilinen yollar üzerinden fiziksel olarak seçilen frontier state'ine gider, successor'ları gözlemler

ve yenilerini kuyruğa ekler. Sonlu branching factor altında her sonlu derinlik state'i sonunda ziyaret edildiği için complete olur.

$(1,-1)$, $(1,0)$ 'ın doğrudan successor'ıdır. Ajan $(0,0) \rightarrow (1,0)$ gittikten sonra successor'ları gözlemlediğinde $(1,-1)$ 'i görüp doğrudan ona geçebilir. Bu durumda gerçek hareket rotası $(0,0) \rightarrow (1,0) \rightarrow (1,-1)$ olur.

Temel Kavramlar Sözlüğü

Terim	Türkçe açıklama
Objective function	Amaç fonksiyonu; state'in ne kadar iyi olduğunu ölçer.
Local / global optimum	Yakın çevrenin en iyisi / tüm uzayın en iyisi.
Plateau / shoulder / ridge	Yerel arama manzarasındaki düz alan, çıkışlı düz alan ve sırt.
Sideways move	Aynı değere sahip komşuya geçiş.
Random restart	Arama takılırsa yeni rastgele başlangıçtan tekrar başlama.
Simulated annealing	Başlangıçta kötü hamleleri daha sık kabul edip zamanla soğuyan stokastik optimizasyon.
Local beam search	Aynı anda k state tutan yerel arama.
Fitness / crossover / mutation	GA'da uygunluk, ebeveyn parçalarını birleştirme ve rastgele değişim.
Gradient / Hessian	Birinci türev yön vektörü / ikinci türev matrisi.
Contingency plan	Olası sonuç/percept'e göre dallanan koşullu plan.
OR node / AND node	Ajanın bir action seçtiği düğüm / çevrenin tüm sonuçlarının ele alınması gereken düğüm.
Belief state	Ajanın içinde bulunması mümkün fiziksel state'ler kümesi.
Prediction / update	Action sonrası olası state'leri tahmin etme / percept ile tutarsız state'leri eleme.
Sensorless / conformant	Percept'in hiç bilgi vermediği problem.
Localization	Ajanın harita üzerindeki kendi konumunu belirlemesi.
Online search	Arama, action ve gözlemi gerçek zamanlı iç içe yürütme.
Competitive ratio	Online gerçek maliyetin önceden bilen optimal maliyete oranı.
Safely explorable	Her erişilebilir state'ten bir goal'a ulaşılabilen state space.
Random walk	Her adımda rastgele action seçen yürüyüş.
LRTA*	H(s) maliyet tahminlerini hareket ederken öğrenen gerçek zamanlı arama.

Ders Öncesi Hızlı Tekrar

- Hill climbing neden local optimumda takılır ve random restart bunu nasıl azaltır?
- Simulated annealing'de sıcaklık T yüksekken ve düşükken davranış nasıl değişir?
- Local beam search ile k bağımsız random restart arasındaki bilgi paylaşımı farkı nedir?
- Genetic algorithm'da crossover neden temsil biçimine bağlıdır?
- Nondeterministic problemde çözüm neden tek bir action sequence olamaz?
- AND ve OR node arasındaki fark nedir?
- Belief state fiziksel state'ten nasıl farklıdır?
- Prediction neden belirsizliği büyütebilir, update neden küçültür?
- Online search'te RESULT(s,a) neden deney yapmadan bilinmez?
- LRTA* local minimumdan hangi öğrenme mekanizmasıyla çıkar?